Tables and Geographic Data

Maithreyi Gopalan

Week 8

Agenda

- Refresher: Flexdashboards and Quarto Blog Deployment
- Tables
  - {gt}
  - {kableExtra}
  - {reactable}
- Geographic data
  - Vector/raster data
  - Producing basic maps
  - Geospatial ecosystem

Refresher

Flexdashboards and Quarto Blog Deployment

What is a Flexdashboard?

- R Markdown format for building dashboard layouts — no Shiny required for static versions
- Install: `install.packages("flexdashboard")`
- New file: **File → New File → R Markdown → From Template → Flex Dashboard**
- Knit to HTML — deploy like any other HTML file

```
---
title: "My Dashboard"
output:
  flexdashboard::flex_dashboard:
    orientation: columns
    vertical_layout: fill
---
```

Flexdashboard Layout Basics

Columns and rows are defined with level 2 headers and dashes:

```
Column {data-width=650}
-----------------------------------

### Chart A
< r code >

Column {data-width=350}
-----------------------------------

### Chart B
< r code >

### Chart C
< r code >
```

- ### creates a new panel — panels split space evenly within a column

Rows Instead of Columns

```
output:
  flexdashboard::flex_dashboard:
    orientation: rows
```

```
Row {data-height=500}
-------------------------------------

### Chart A

Row {data-height=300}
-------------------------------------

### Chart B

### Chart C
```

Multiple Pages

```
# Page 1

Column {data-width=650}
-------------------------------------
### Chart A

# Page 2

Column
-------------------------------------
### Chart B
```

Level 1 headers (#) create new pages in the nav bar.

Tabsets

```
Column {.tabset data-width=650}
-------------------------------------

### Chart 1
< r code >

### Chart 2
< r code >

### Data Table
< r code >
```

No comma between multiple column arguments:

- ✅ `Column {.tabset data-width=650}`
- ❌ `Column {.tabset, data-width=650}`

Sidebar

```
Sidebar {.sidebar}
====================================

Describe the dashboard here. Supports
*markdown**, [links](http://example.com), etc.
```

Use `====` (not `----`) to keep the sidebar across multiple pages.

Adding Interactivity

For `plotly` and `reactable` — no Shiny needed:

```
library(plotly)
p <- ggplot(mpg, aes(displ, cty)) +
  geom_point() +
  geom_smooth()

ggplotly(p)
```

For full Shiny interactivity, add `runtime: shiny` to YAML and use `render*` functions.

Deploying a Flexdashboard

Since it renders to HTML, deploy anywhere that hosts HTML:

- **GitHub Pages** — push the `.html` file to your repo, enable Pages
- **Netlify** — drag and drop the HTML file at netlify.com/drop
- **shinyapps.io** — required if using `runtime: shiny`

For GitHub Pages, simplest approach:

```
# Keep your dashboard in the root or docs/ folder
# Ensure that its renamed to index.html
# Enable Pages in repo Settings → Pages → /root or /docs
```

Quarto Blog

Quick Deployment Refresher

Quarto Blog: The Key Files

```
my-blog/
├── _quarto.yml      # site settings — theme, navbar, output-dir
├── index.qmd        # listing page (your home page)
├── about.qmd        # about/bio page
├── posts/           # one subfolder per post
│   └── my-post/
│       └── index.qmd
├── styles.css       # CSS overrides
└── docs/            # built site — this gets pushed to GitHub
```

`output-dir: docs` in `_quarto.yml` is what makes GitHub Pages work.

_quarto.yml Essentials

```yaml
project:
  type: website
  output-dir: docs

website:
  title: "My Blog"
  site-url: https://USERNAME.github.io/my-blog/
  navbar:
    right:
      - about.qmd
      - icon: github
        href: https://github.com/USERNAME

format:
  html:
    theme: flatly
    css: styles.css
    toc: true

execute:
  freeze: auto
  warning: false
```

Writing a Post

Create `posts/my-post/index.qmd`:

```
---
title: "My Post Title"
author: "Maithreyi Gopalan"
date: "2026-02-18"
categories: [R, education policy]
image: "thumbnail.png"
draft: false
---
```

Write content below the `---`. Use `#|` for chunk options:

```

```

Render and Preview

```
# Live preview with hot reload
quarto preview

# Build the full site to /docs
quarto render
```

`freeze: auto` means posts only re-run when source changes — commit `_freeze/` to GitHub!

Deploying to GitHub Pages

1. Run `quarto render` — builds site into `docs/`
2. Commit everything including `docs/`:

```
git add .
git commit -m "Build site"
git push
```

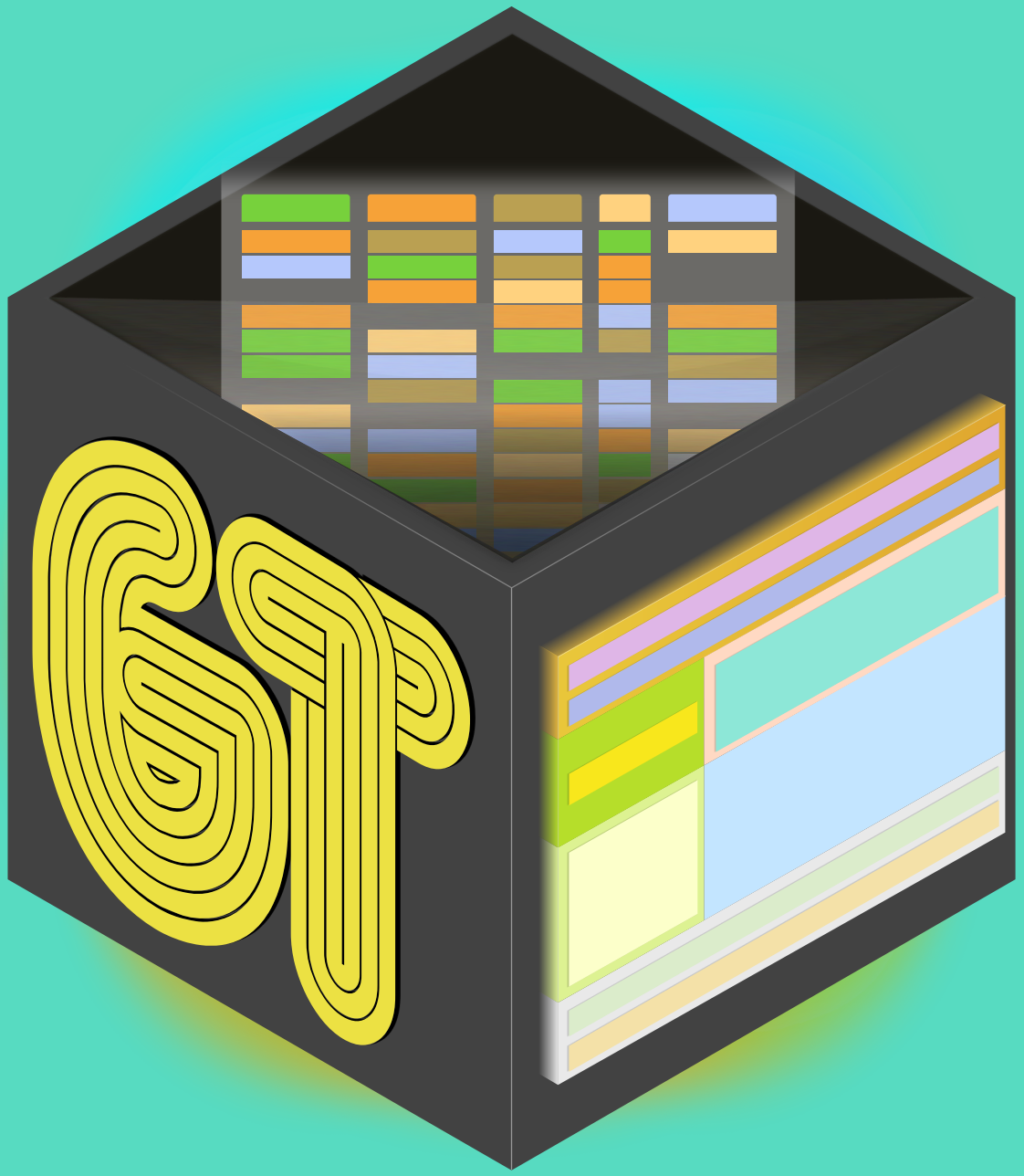
1. On GitHub: **Settings → Pages → Branch: main, Folder: /docs → Save**

Your site goes live at
`https://USERNAME.github.io/REPO/`

Troubleshooting Deployment

Problem	Fix
Posts missing from listing	Check `draft: false` and post is in `posts/`
Site not updating	Confirm `docs/` is committed, not in `.gitignore`
YAML errors	Remove `#` inline comments from YAML blocks
Push rejected	Use a PAT token, not your GitHub password
Fonts not loading	`@import` must be first line of `styles.css`

Tables



{gt} Overview

- Pipe-oriented — works naturally with `%>%`
- Beautiful tables with minimal code
- Spanner heads and grouping built in
- Renders to HTML and PDF seamlessly

Probably the best package for static tables. {kableExtra} is also excellent — fewer people know `gt`, which is why we cover it here.

Install

```
install.packages("gt")
```

The hard part

- Getting your data into the right shape for a table
- Use your `pivot_*` skills

```
library(fivethirtyeight)
flying
```

```
## # A tibble: 1,040 × 27
##    respondent_id gender age   height   children_under_18 household_income    ed
##            <dbl> <chr>  <ord> <ord>    <lgl>             <ord>                <o
##  1    3436139758 <NA>   <NA>   <NA>    NA                <NA>                 <N
##  2    3434278696 Male   30-44 "6'3\""  TRUE              <NA>                 Gr
##  3    3434275578 Male   30-44 "5'8\""  FALSE             $100,000 - $149,999  Ba
##  4    3434268208 Male   30-44 "5'11\"" FALSE             $0 - $24,999         Ba
##  5    3434250245 Male   30-44 "5'7\""  FALSE             $50,000 - $99,999    Ba
##  6    3434245875 Male   30-44 "5'9\""  TRUE              $25,000 - $49,999    Gr
##  7    3434235351 Male   30-44 "6'2\""  TRUE              <NA>                 So
##  8    3434218031 Male   30-44 "6'0\""  TRUE              $0 - $24,999         Ba
##  9    3434213681 <NA>   <NA>  "6'0\""  TRUE              <NA>                 <N
## 10    3434172894 Male   30-44 "5'6\""  FALSE             $0 - $24,999         Ba
## # ℹ 1,030 more rows
## # ℹ 15 more variables: recline_eliminate <lgl>, switch_seats_friends <ord>, swi
## #   unruly_child <ord>, two_arm_rests <chr>, middle_arm_rest <chr>, shade <chr>
```

```
flying %>%
  count(gender, age, recline_frequency)
```

```
## # A tibble: 53 × 4
##    gender age   recline_frequency       n
##    <chr>  <ord> <ord>               <int>
##  1 Female 18-29 Never                  24
##  2 Female 18-29 Once in a while        36
##  3 Female 18-29 About half the time    10
##  4 Female 18-29 Usually                13
##  5 Female 18-29 Always                 10
##  6 Female 18-29 <NA>                   19
##  7 Female 30-44 Never                  21
##  8 Female 30-44 Once in a while        25
##  9 Female 30-44 About half the time    22
## 10 Female 30-44 Usually                22
## # ℹ 43 more rows
```

```r
smry <- flying %>%
  count(gender, age, recline_frequency) %>%
  drop_na(age, recline_frequency) %>%
  pivot_wider(
    names_from = "age",
    values_from = "n"
  )
smry
```

```
## # A tibble: 10 × 6
##    gender recline_frequency   `18-29` `30-44` `45-60` `> 60`
##    <chr>  <ord>                 <int>   <int>   <int>  <int>
##  1 Female Never                    24      21      19     23
##  2 Female Once in a while          36      25      30     36
##  3 Female About half the time      10      22      18     17
##  4 Female Usually                  13      22      26     28
##  5 Female Always                   10      21      29     12
##  6 Male   Never                    24      17      20     18
##  7 Male   Once in a while          19      39      40     29
##  8 Male   About half the time      11      11      16     11
##  9 Male   Usually                  14      30      15     27
## 10 Male   Always                   11      14      21     14
```

Turn into a table

```
library(gt)
smry %>%
  gt()
```

Note: these may look slightly different on slides

The way they render locally is how they'll appear in a standard R Markdown or Quarto file.

gender	recline_frequency	18-29	30-44	45-60	> 60
Female	Never	24	21	19	23
Female	Once in a while	36	25	30	36
Female	About half the time	10	22	18	17
Female	Usually	13	22	26	28
Female	Always	10	21	29	12
Male	Never	24	17	20	18
Male	Once in a while	19	39	40	29
Male	About half the time	11	11	16	11
Male	Usually	14	30	15	27
Male	Always	11	14	21	14

Add gender as a grouping variable

```
smry %>%
  group_by(gender) %>%
  gt()
```

recline_frequency	18-29	30-44	45-60	> 60
Female				
Never	24	21	19	23
Once in a while	36	25	30	36
About half the time	10	22	18	17
Usually	13	22	26	28
Always	10	21	29	12
Male				
Never	24	17	20	18
Once in a while	19	39	40	29
About half the time	11	11	16	11
Usually	14	30	15	27
Always	11	14	21	14

Add a spanner head

```
smry %>%
  group_by(gender) %>%
  gt() %>%
  tab_spanner(
    label = "Age Range",
    columns = c(`18-29`, `30-44`, `45-60`, `> 60`)
  )
```

	Age Range			
recline_frequency	18-29	30-44	45-60	> 60
Female				
Never	24	21	19	23
Once in a while	36	25	30	36
About half the time	10	22	18	17
Usually	13	22	26	28
Always	10	21	29	12
Male				
Never	24	17	20	18
Once in a while	19	39	40	29
About half the time	11	11	16	11
Usually	14	30	15	27
Always	11	14	21	14

Change column names

```
smry %>%
  group_by(gender) %>%
  gt() %>%
  tab_spanner(
    label = "Age Range",
    columns = c(`18-29`, `30-44`, `45-60`, `> 60`)
  ) %>%
  cols_label(recline_frequency = "Recline")
```

	Age Range			
Recline	18-29	30-44	45-60	> 60
Female				
Never	24	21	19	23
Once in a while	36	25	30	36
About half the time	10	22	18	17
Usually	13	22	26	28
Always	10	21	29	12
Male				
Never	24	17	20	18
Once in a while	19	39	40	29
About half the time	11	11	16	11
Usually	14	30	15	27
Always	11	14	21	14

Align columns

```
smry %>%
  group_by(gender) %>%
  gt() %>%
  tab_spanner(
    label = "Age Range",
    columns = c(`18-29`, `30-44`, `45-60`, `> 60`)
  ) %>%
  cols_label(recline_frequency = "Recline") %>%
  cols_align(align = "left",
             columns = recline_frequency)
```

	Age Range			
Recline	18-29	30-44	45-60	> 60
Female				
Never	24	21	19	23
Once in a while	36	25	30	36
About half the time	10	22	18	17
Usually	13	22	26	28
Always	10	21	29	12
Male				
Never	24	17	20	18
Once in a while	19	39	40	29
About half the time	11	11	16	11
Usually	14	30	15	27
Always	11	14	21	14

Add a title

```
smry %>%
  group_by(gender) %>%
  gt() %>%
  tab_spanner(
    label = "Age Range",
    columns = c(`18-29`, `30-44`, `45-60`, `> 60`)
  ) %>%
  cols_label(recline_frequency = "Recline") %>%
  cols_align(align = "left", columns = recline_frequency) %>%
  tab_header(
    title = "Airline Passengers",
    subtitle = "Leg space is limited, what do you do?"
  )
```

Airline Passengers

Leg space is limited, what do you do?

	Age Range			
Recline	18-29	30-44	45-60	> 60
Female				
Never	24	21	19	23
Once in a while	36	25	30	36
About half the time	10	22	18	17
Usually	13	22	26	28
Always	10	21	29	12
Male				
Never	24	17	20	18
Once in a while	19	39	40	29
About half the time	11	11	16	11
Usually	14	30	15	27
Always	11	14	21	14

Format columns

```
smry %>%
  mutate(across(c(`18-29`, `30-44`, `45-60`, `> 60`),
                ~.x / 100)) %>%
  group_by(gender) %>%
  gt() %>%
  tab_spanner(
    label = "Age Range",
    columns = c(`18-29`, `30-44`, `45-60`, `> 60`)
  ) %>%
  fmt_percent(
    columns = c(`18-29`, `30-44`, `45-60`, `> 60`),
    decimals = 0
  ) %>%
  cols_label(recline_frequency = "Recline") %>%
  cols_align(align = "left", columns = recline_frequency) %>%
  tab_header(
    title = "Airline Passengers",
    subtitle = "Leg space is limited, what do you do?"
  )
```

Airline Passengers

Leg space is limited, what do you do?

	Age Range			
Recline	18-29	30-44	45-60	> 60
Female				
Never	24%	21%	19%	23%
Once in a while	36%	25%	30%	36%
About half the time	10%	22%	18%	17%
Usually	13%	22%	26%	28%
Always	10%	21%	29%	12%
Male				
Never	24%	17%	20%	18%
Once in a while	19%	39%	40%	29%
About half the time	11%	11%	16%	11%
Usually	14%	30%	15%	27%
Always	11%	14%	21%	14%

Add a source note

```
... %>%
  tab_source_note(
    source_note = md("Data from [fivethirtyeight](https://fiveth
  )
```

Airline Passengers

Leg space is limited, what do you do?

	Age Range			
Recline	18-29	30-44	45-60	> 60
Female				
Never	24%	21%	19%	23%
Once in a while	36%	25%	30%	36%
About half the time	10%	22%	18%	17%
Usually	13%	22%	26%	28%
Always	10%	21%	29%	12%
Male				
Never	24%	17%	20%	18%
Once in a while	19%	39%	40%	29%
About half the time	11%	11%	16%	11%
Usually	14%	30%	15%	27%
Always	11%	14%	21%	14%

Data from fivethirtyeight

Color cells

```
... %>%
  data_color(
    columns = c(`18-29`, `30-44`, `45-60`, `> 60`),
    fn = scales::col_numeric(
      palette = c("#FFFFFF", "#FF0000"),
      domain = NULL
    )
  ) %>%
  ...
```

Airline Passengers

Leg space is limited, what do you do?

	Age Range			
Recline	18-29	30-44	45-60	> 60
Female				
Never	24%	21%	19%	23%
Once in a while	36%	25%	30%	36%
About half the time	10%	22%	18%	17%
Usually	13%	22%	26%	28%
Always	10%	21%	29%	12%
Male				
Never	24%	17%	20%	18%
Once in a while	19%	39%	40%	29%
About half the time	11%	11%	16%	11%
Usually	14%	30%	15%	27%
Always	11%	14%	21%	14%

Data from fivethirtyeight

What else?

- Lots more it can do — see the website

Thomas Mock does great work with tables and often shares tutorials (e.g., embedding custom features, functions and themes, 10 table rules).

A few other table options

kableExtra

A few quick examples

Make sure to specify `results = "asis"` in your chunk options.

```
library(knitr)
library(kableExtra)
dt <- mtcars[1:5, 1:6]
kable(dt) %>%
  kable_styling("striped") %>%
  column_spec(5:7, bold = TRUE
```

	mpg	cyl	disp	hp	drat	wt
Mazda RX4	21.0	6	160	**110**	**3.90**	**2.620**
Mazda RX4 Wag	21.0	6	160	**110**	**3.90**	**2.875**
Datsun 710	22.8	4	108	**93**	**3.85**	**2.320**
Hornet 4 Drive	21.4	6	258	**110**	**3.08**	**3.215**
Hornet Sportabout	18.7	8	360	**175**	**3.15**	**3.440**

```
kable(dt) %>%
  kable_styling("striped") %>%
  column_spec(5:7, bold = TRUE) %>%
  row_spec(c(2, 4),
           bold = TRUE,
           color = "#EFF3F7",
           background = "#71B0DE")
```

	mpg	cyl	disp	hp	drat	wt
Mazda RX4	21.0	6	160	**110**	**3.90**	**2.620**
Mazda RX4 Wag	**21.0**	**6**	**160**	**110**	**3.90**	**2.875**
Datsun 710	22.8	4	108	**93**	**3.85**	**2.320**
Hornet 4 Drive	**21.4**	**6**	**258**	**110**	**3.08**	**3.215**
Hornet Sportabout	18.7	8	360	**175**	**3.15**	**3.440**

```
kable(dt) %>%
  kable_styling("striped", full_width = FALSE) %>%
  pack_rows(
    "Group 1", 1, 3,
    label_row_css = "background-color: #666; color: #fff;"
  ) %>%
  pack_rows(
    "Group 2", 4, 5,
    label_row_css = "background-color: #666; color: #fff;"
  )
```

	mpg	cyl	disp	hp	drat	wt
Group 1						
Mazda RX4	21.0	6	160	110	3.90	2.620
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875
Datsun 710	22.8	4	108	93	3.85	2.320
Group 2						
Hornet 4 Drive	21.4	6	258	110	3.08	3.215
Hornet Sportabout	18.7	8	360	175	3.15	3.440

kableExtra wrapup

Many other options — see the documentation. Works well for both PDF and HTML.

What about Microsoft Word? Use {flextable} — it's specifically designed for Word output.

Many others

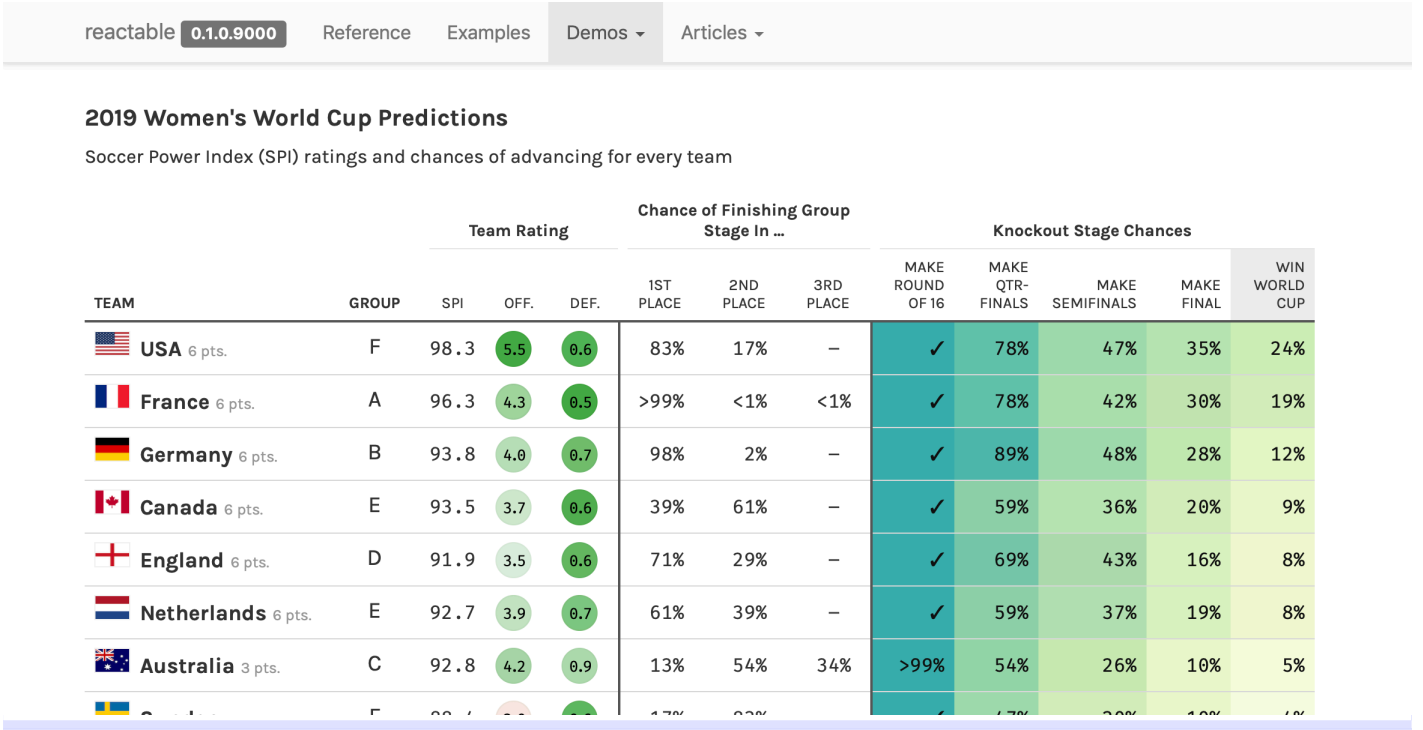
- huxtable
- formattable
- DT
- rhandsontable

  Particularly helpful for modeling output
- modelsummary
- stargazer
- pixiedust

  For descriptives
- gtsummary

reactable

My favorite for interactive tables



Works great with **shiny** too

Penguins data

```
library(palmerpenguins)
library(reactable)
reactable(penguins)
```

species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	se
Adelie	Torgersen	39.1	18.7	181	3750	ma
Adelie	Torgersen	39.5	17.4	186	3800	fe
Adelie	Torgersen	40.3	18	195	3250	fe
Adelie	Torgersen					
Adelie	Torgersen	36.7	19.3	193	3450	fe
Adelie	Torgersen	39.3	20.6	190	3650	ma
Adelie	Torgersen	38.9	17.8	181	3625	fe
Adelie	Torgersen	39.2	19.6	195	4675	ma
Adelie	Torgersen	34.1	18.1	193	3475	
Adelie	Torgersen	42	20.2	190	4250	

1–10 of 344 rows Previous **1** 2 3 4 5 ... 35 Next

Rename columns

```
penguins %>%
  reactable(
    columns = list(
      bill_length_mm = colDef(name = "Bill Length (mm)"),
      bill_depth_mm  = colDef(name = "Bill Depth (mm)")
    )
  )
```

species	island	Bill Length (mm)	Bill Depth (mm)	flipper_length_mm	body_mass_g	se
Adelie	Torgersen	39.1	18.7	181	3750	ma
Adelie	Torgersen	39.5	17.4	186	3800	fe
Adelie	Torgersen	40.3	18	195	3250	fe
Adelie	Torgersen					
Adelie	Torgersen	36.7	19.3	193	3450	fe
Adelie	Torgersen	39.3	20.6	190	3650	ma
Adelie	Torgersen	38.9	17.8	181	3625	fe
Adelie	Torgersen	39.2	19.6	195	4675	ma
Adelie	Torgersen	34.1	18.1	193	3475	
Adelie	Torgersen	42	20.2	190	4250	

1–10 of 344 rows Previous **1** 2 3 4 5 ... 35 Next

Or use a function

```
penguins %>%
  reactable(
    defaultColDef = colDef(
      header = function(x) str_to_title(gsub("_", " ", x))
    )
  )
```

Species	Island	Bill Length Mm	Bill Depth Mm	Flipper Length Mm	Body Mass G	Se
Adelie	Torgersen	39.1	18.7	181	3750	ma
Adelie	Torgersen	39.5	17.4	186	3800	fe
Adelie	Torgersen	40.3	18	195	3250	fe
Adelie	Torgersen					
Adelie	Torgersen	36.7	19.3	193	3450	fe
Adelie	Torgersen	39.3	20.6	190	3650	ma
Adelie	Torgersen	38.9	17.8	181	3625	fe
Adelie	Torgersen	39.2	19.6	195	4675	ma
Adelie	Torgersen	34.1	18.1	193	3475	
Adelie	Torgersen	42	20.2	190	4250	

1–10 of 344 rows Previous **1** 2 3 4 5 ... 35 Next

Add filter, search, pagination

```
reactable(penguins, filterable = TRUE)
reactable(penguins, searchable = TRUE)
reactable(penguins, defaultPageSize = 3)
reactable(penguins, defaultPageSize = 3, paginationType = "jump")
```

Search

species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	se
Adelie	Torgersen	39.1	18.7	181	3750	ma
Adelie	Torgersen	39.5	17.4	186	3800	fe
Adelie	Torgersen	40.3	18	195	3250	fe
Adelie	Torgersen					
Adelie	Torgersen	36.7	19.3	193	3450	fe

1–5 of 344 rows

Previous **1** 2 3 4 5 ... 69 Next

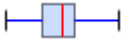
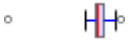
Grouping and aggregation

```
penguins %>%
  reactable(
    groupBy = c("species", "island"),
    columns = list(
      bill_length_mm = colDef(aggregate = "mean",
                              format = colFormat(digits = 2))
    )
  )
```

species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	se
▶ Adelie (3)		38.79				
▶ Gentoo (1)		47.50				
▶ Chinstrap (1)		48.83				

Sparklines

```
library(sparkline)
table_data <- penguins %>%
  group_by(species) %>%
  summarize(bill_length = list(bill_length_mm)) %>%
  mutate(boxplot = NA, sparkline = NA)

table_data %>%
  reactable(
    columns = list(
      bill_length = colDef(cell = function(value) {
        sparkline(value, type = "bar")
      }),
      boxplot = colDef(cell = function(value, index) {
        sparkline(table_data$bill_length[[index]], type = "box")
      }),
      sparkline = colDef(cell = function(value, index) {
        sparkline(table_data$bill_length[[index]])
      })
    )
  )
```

species	bill_length	boxplot	sparkline
Adelie			
Chinstrap			
Gentoo			

Lots more!

The goal today isn't to teach you everything — it's to show you what's possible. Check out the documentation for more.

Also check out {reactablefmtr} for easier use and amazing visual extensions!

Handling Data for Deployment

Flexdashboards and Quarto Blogs

The Core Question

When you embed a widget or dashboard in a Quarto blog — where does the **data** live?

Four options, each with tradeoffs:

- Self-contained HTML
- Committed data folder
- Inline base64 encoding
- Remote data source

The right choice depends on **data size** and **sensitivity**

Option 1: Self-Contained HTML

Add one line to your flexdashboard YAML:

```
output:
  flexdashboard::flex_dashboard:
    self_contained: true
```

- Bundles data, widgets, and CSS into a **single HTML file**
- Just commit and push that one file — no broken paths
- ✅ Best for: small datasets, maximum simplicity
- ❌ Avoid if: file size gets large (>10MB)

Option 2: Commit a Data Folder

Keep data alongside your built site:

```
docs/
├── my-dashboard.html
└── data/
    └── my_data.csv
```

Read with a relative path in your Rmd:

```
df <- read_csv("data/my_data.csv")
```

- Commit everything including `data/` and push to GitHub
- ✅ Best for: medium datasets you want to reuse across posts
- ❌ Avoid if: data is sensitive or changes frequently

Option 3: Remote Data Source

Pull live from an API, Google Sheets, or database at render time:

```
library(googlesheets4)
df <- read_sheet("your-sheet-id")

# or from tidycensus, an API, etc.
df <- get_acs(geography = "tract", ...)
```

- Nothing sensitive ever touches GitHub
- Data stays up to date without re-rendering
- ✅ Best for: sensitive, large, or frequently updated data
- Add `data/` to `.gitignore` if caching locally

Embedding Widgets: Quarto Blog

In a Quarto blog post, widgets work **natively** — no extra steps:

```
<div class="reactable html-widget html-fill-item" id="htmlwidget-
<script type="application/json" data-for="htmlwidget-14f8f7ac45a(
```

- `quarto render` automatically bundles widget dependencies into `docs/`
- Works for: `reactable`, `plotly`, `leaflet`, `mapview`, `DT`, and more
- Self-contained per post — just push `docs/` as usual

```
docs/
├── index.html
└── posts/
└── my-post/
    └── index.html   ← widget dependencies bundled in here
```

Embedding Widgets: Flexdashboard

In a flexdashboard, widgets also embed **inline by default** when knitting:

````
### My Interactive Table

```r
library(reactable)
reactable(penguins,
          searchable = TRUE,
          filterable = TRUE)
```
````

- No `saveWidget()` needed — knitting produces one self-contained HTML
- Works for all htmlwidgets: `plotly`, `leaflet`, `reactable`, `DT`, etc.
- For Shiny-powered widgets, add `runtime: shiny` to

The Exception: Xaringan Slides

Xaringan does **not** support HTML widgets natively — widgets render blank:

```
# This will NOT show in xaringan slides
reactable(penguins)
```

The fix — save each widget and embed via `<iframe>`:

```
# Save widget to file
saveWidget(reactable(penguins),
           "widgets/rt_basic.html",
           selfcontained = TRUE)
```

Then in the slide:

```
<iframe src="widgets/rt_basic.html"
        width="100%" height="450px"
```

Which Option Should You Use?

Situation	Approach
Quarto blog post	Use widget directly — Quarto handles it
Flexdashboard	Use widget directly — embeds on knit
Xaringan slides	`saveWidget()` + `<iframe>` + commit `widgets/`
Small data, want simplicity	`self_contained: true` in YAML
Medium data, reused across posts	Commit `data/` folder
Sensitive or large data	Remote source + `.gitignore`

Data Viz in the wild

Cheyna and Ramtin

- Kyla and Isha on deck for next week

Geographic data

First — a disclaimer

- We're *only* talking about **visualizing** geographic data, not analyzing it
- There's SO MUCH we won't get to
- Today is an intro — hopefully you'll feel comfortable with the basics

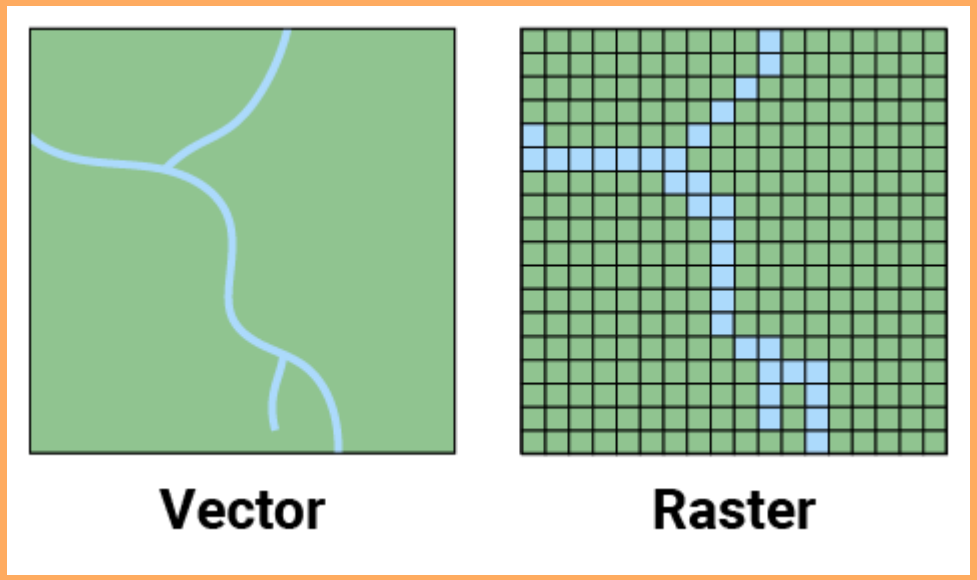
Learning objectives

- Know the difference between vector and raster data
- Be able to produce basic maps using a variety of tools
- Be able to obtain different types of geographic data from a few different places
- Be able to produce basic interactive maps
- Understand the basics of the R geospatial ecosystem

Today is partially about content and partially about exposure

Where to learn more

Geocomputation with R



Vector versus

Image from Zev Ross

Vector data

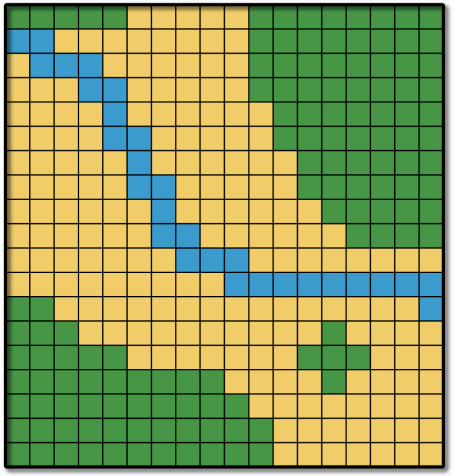
- Points, lines, and polygons
- Can easily include non-spatial attributes (e.g., population within a polygon)
- Come in the form of shapefiles (`.shp`), GeoJSON, or frequently in R packages

This is what we'll focus on today

Tends to be most relevant for social science research questions

Raster data

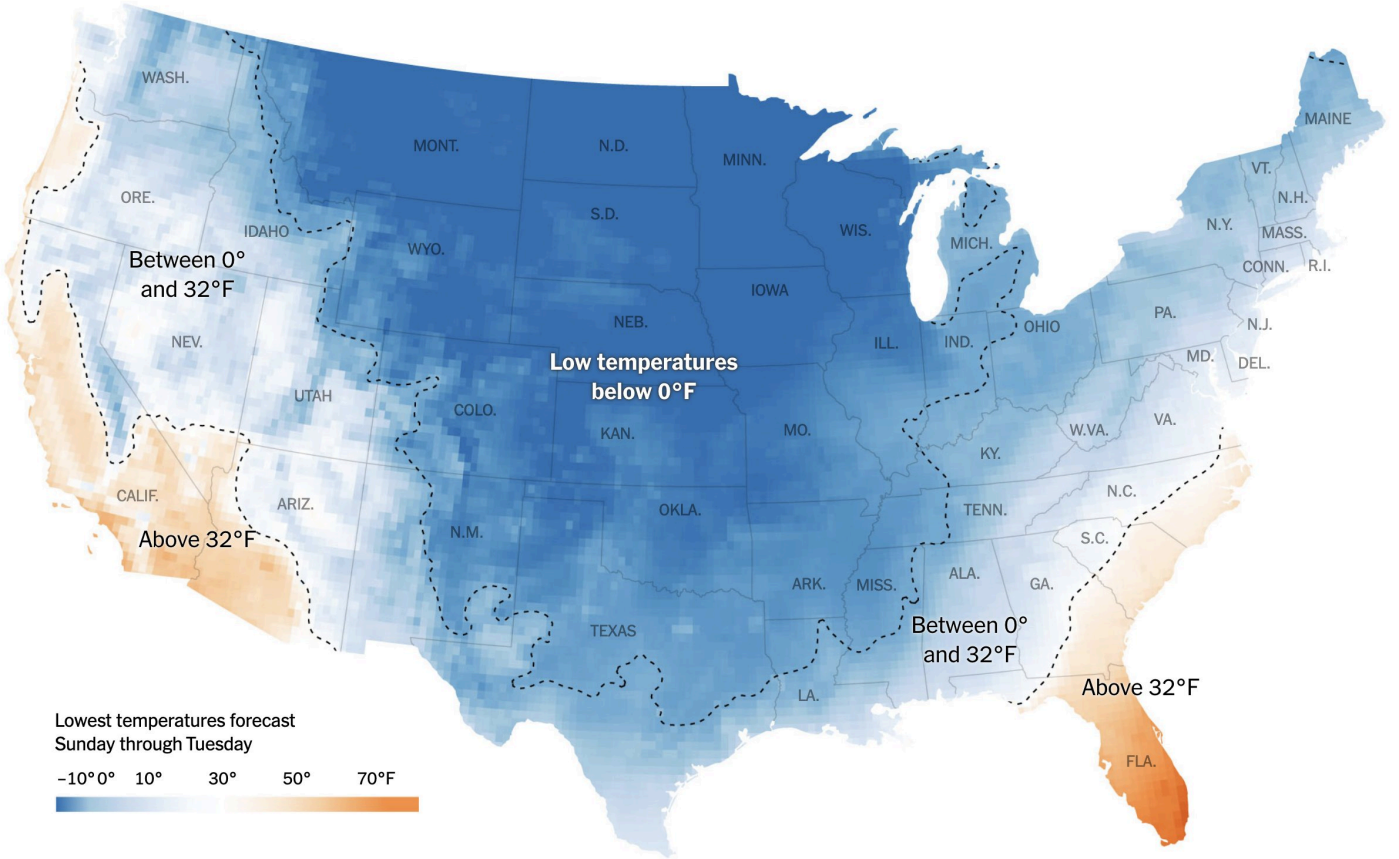
- Divides space into a grid
- Each square (pixel) gets a value



Common formats include images, satellite data, and remote sensing data.

Can be useful in social science to show things like population density or temperature grids.

Example



source

The #rspatial ecosystem

- {sf} — simple features, the standard for vector data
- {terra} — modern replacement for {raster}
- {ggplot2} — `geom_sf()` for plotting sf objects
- {tmap} — thematic map package
- {mapview} — quick interactive maps

Goal today

Take you through a basic tour of each of these.

Some specific challenges with geospatial data

- Coordinate reference systems and projections
- List columns (specifically when working with {sf} objects)
- Different geometry types (lines, points, polygons)
- Vector versus raster

Working with spatial data

Two main options:

- `spatialDataFrame` (from the {sp} package) — older, less common now
- **sf data frame** (simple features) — the modern standard, works natively with tidyverse

Use `sf::st_as_sf()` to convert `{sp}` to `{sf}` if needed.

{tigris}

```
library(tigris)
library(sf)
options(tigris_class = "sf")

roads_laneco <- roads("OR", "Lane")
roads_laneco
```

```
## Simple feature collection with 20433 features and 4 fields
## Geometry type: LINESTRING
## Dimension:     XY
## Bounding box:  xmin: -124.1536 ymin: 43.4376 xmax: -121.8078 ymax: 44.29001
## Geodetic CRS:  NAD83
## # A tibble: 20,433 × 5
##    LINEARID      FULLNAME           RTTYP MTFCC
##    <chr>         <chr>              <chr> <chr>
##  1 1102152610459 W Lone Oak Lp      M     S1640 (-123.1256 44.10108, -123.1262
##  2 1102217699747 Sheldon Village Lp M     S1640 (-123.0742 44.07891, -123.073 4
##  3 110458663505  Cottage Hts Lp     M     S1400 (-123.0522 43.7893, -123.0522 4
##  4 1102152615811 Village Plz Lp     M     S1640          (-123.1051 44.08716, -
##  5 110458661289  River Pointe Lp    M     S1400 (-123.0864 44.10306, -123.0852
##  6 1102217699746 Sheldon Village Lp M     S1640 (-123.0723 44.07875, -123.0723
##  7 110458664549  Village Plz Lp     M     S1400 (-123.1053 44.08658, -123.1052
##  8 1102223141058 Carpenter Byp      M     S1400
##  9 1106092829172 State Hwy 126 Bus  S     S1200 (-123.0502 44.04391, -123.0502
```

I/O

Write to disk:

```
write_sf(roads_laneco, here::here("data", "roads_lane.shp"))
```

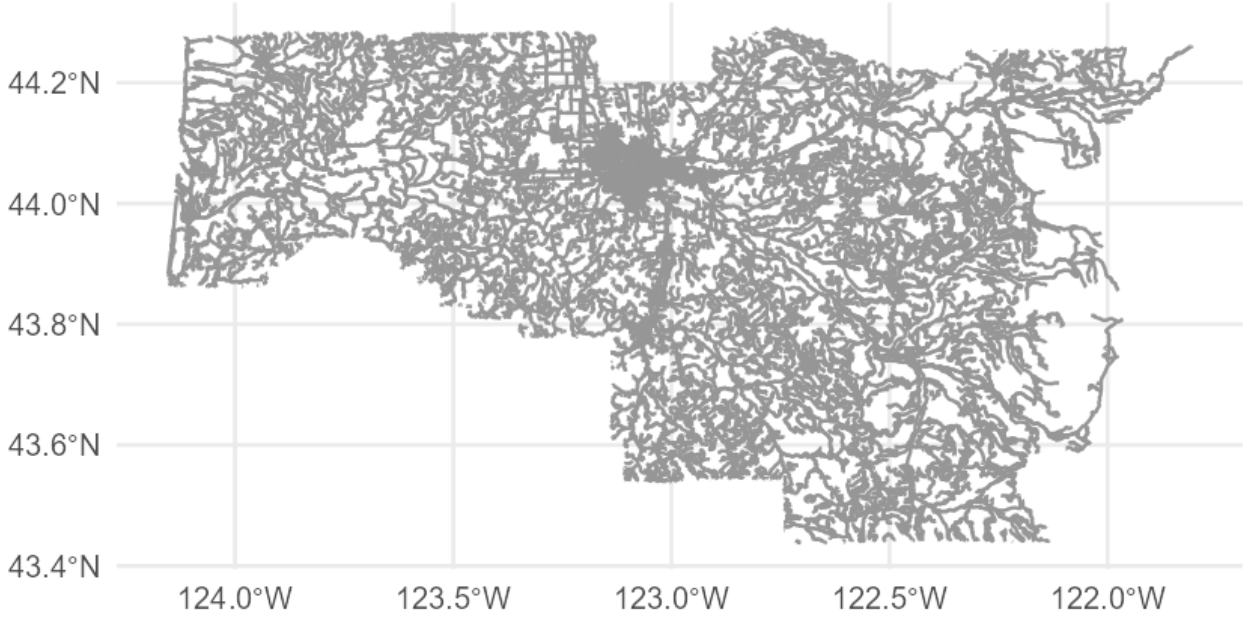
Read back in:

```
roads_laneco <- read_sf(here::here("data", "roads_lane.shp"))
```

{sf} works with ggplot

Use `ggplot2::geom_sf()`

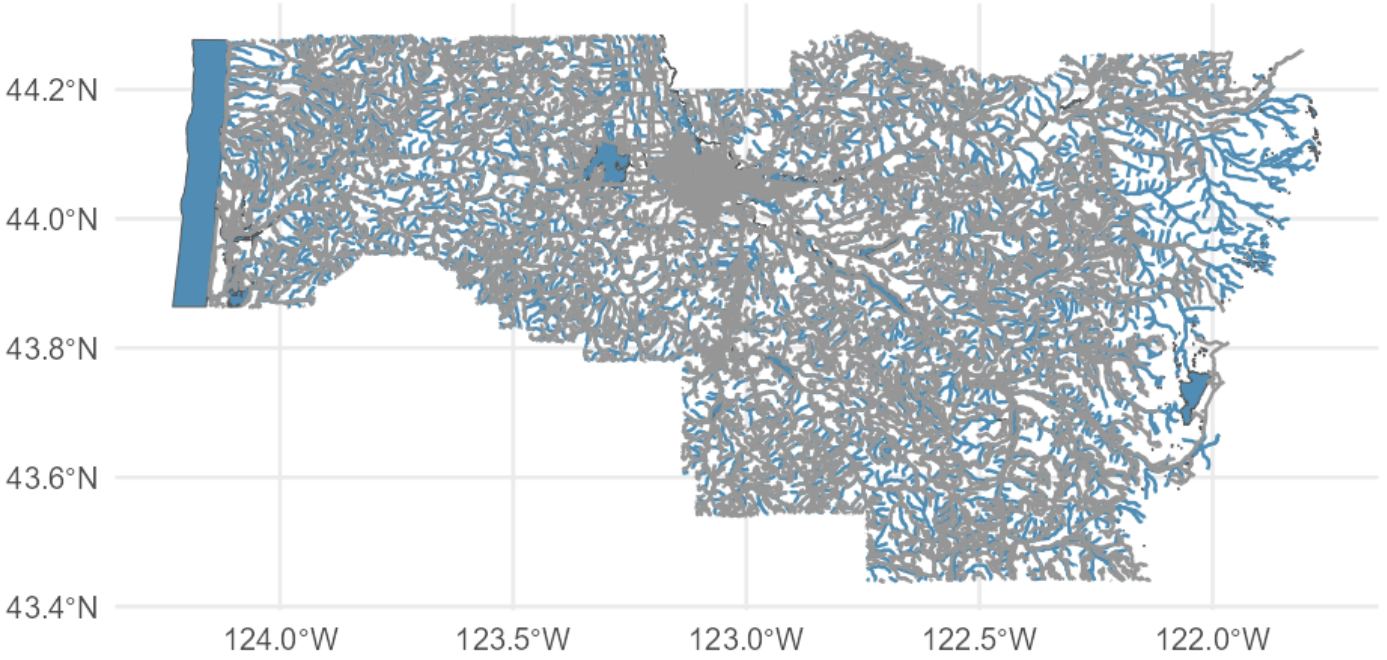
```
ggplot(roads_laneco) +
  geom_sf(color = "gray60")
```



Add water features

```
lakes   <- area_water("OR", "Lane")
streams <- linear_water("OR", "Lane")

ggplot() +
  geom_sf(data = lakes,       fill = "#518FB5") +
  geom_sf(data = streams,    color = "#518FB5") +
  geom_sf(data = roads_laneco, color = "gray60")
```

Note — these functions are all from {tigris}.


44.2°N
44.0°N
43.8°N
43.6°N
43.4°N
124.0°W
123.5°W
123.0°W
122.5°W
122.0°W

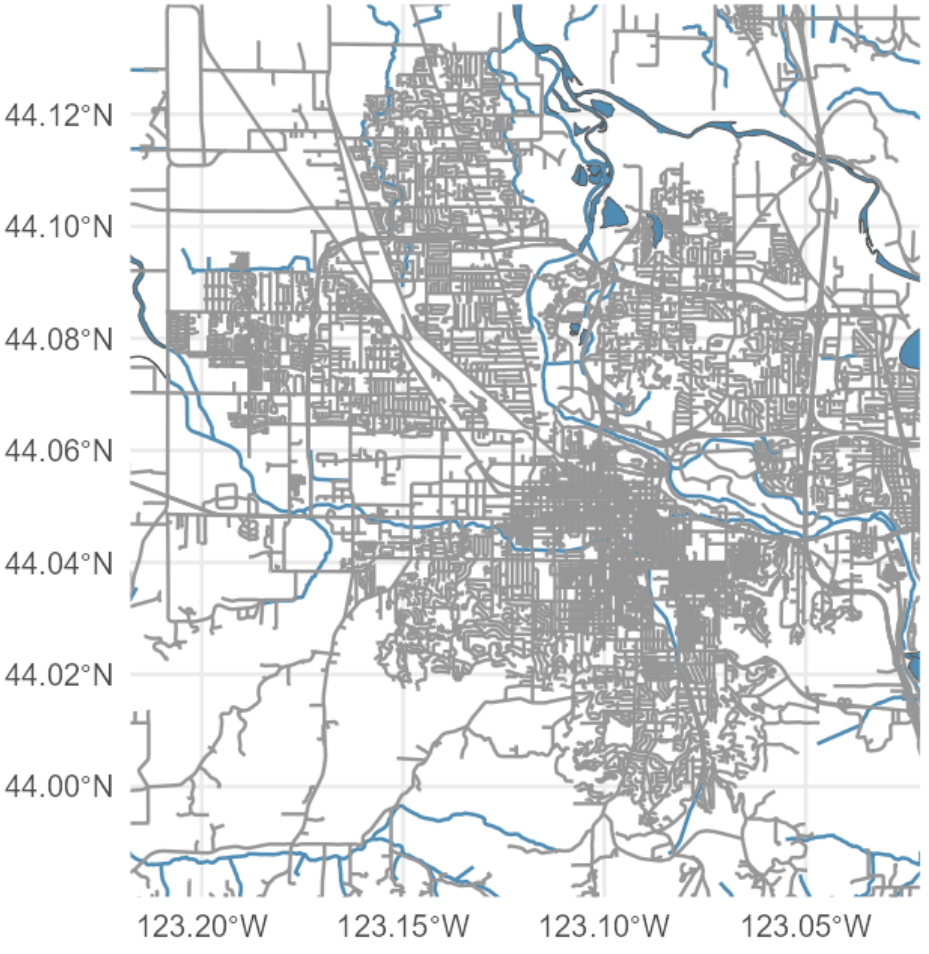
Quick aside: `osmdata`

- Specifically for street-level data from OpenStreetMap

```
bb <- osmdata::getbb("Eugene")
bb
```

```
##          min        max
## x -123.20876 -123.03059
## y   43.98753   44.13232
```

```
ggplot() +
  geom_sf(data = lakes,        fill = "#518FB5") +
  geom_sf(data = streams,      color = "#518FB5", linewidth = 1.2)
  geom_sf(data = roads_laneco, color = "gray60") +
  coord_sf(xlim = bb[1, ], ylim = bb[2, ])
```

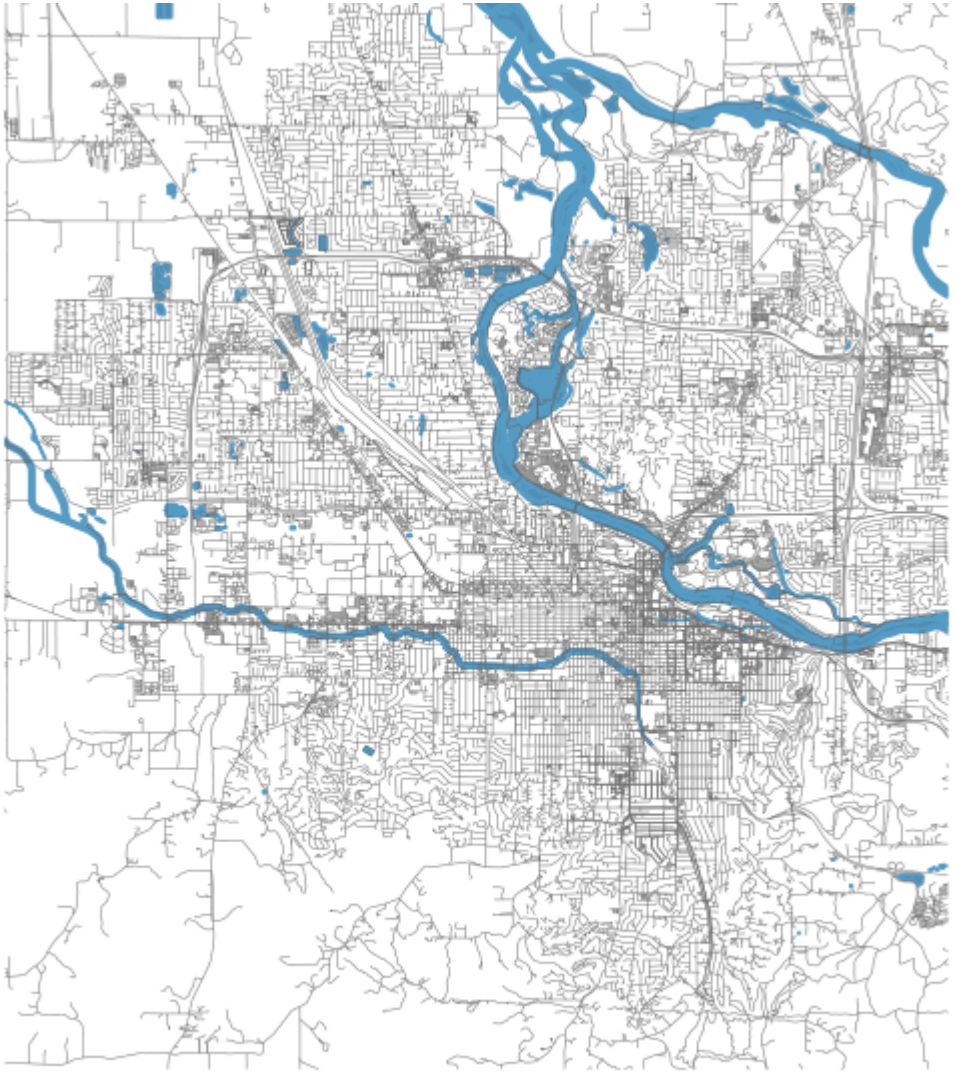

44.12°N
44.10°N
44.08°N
44.06°N
44.04°N
44.02°N
44.00°N
123.20°W
123.15°W
123.10°W
123.05°W

Fully with osmdata

```
library(osmdata)
library(colorspace)

bb <- getbb("Eugene")

roads <- bb %>%
  opq() %>%
  add_osm_feature("highway") %>%
  osmdata_sf()

water <- bb %>%
  opq() %>%
  add_osm_feature("water") %>%
  osmdata_sf()
```

Plot with osmdata

```
ggplot() +
  geom_sf(data = water$osm_multipolygons,
          fill = "#518FB5", color = darken("#518FB5")) +
  geom_sf(data = water$osm_polygons,
          fill = "#518FB5", color = darken("#518FB5")) +
  geom_sf(data = water$osm_lines,
          color = darken("#518FB5")) +
  geom_sf(data = roads$osm_lines,
          color = "gray40", linewidth = 0.2) +
  coord_sf(xlim = bb[1, ], ylim = bb[2, ], expand = FALSE) +
  labs(caption = "Eugene, OR")
```



Eugene, OR

Getting census data with {tidycensus}

Note

You need a Census API key — register at api.census.gov/data/key_signup.html

Then set it with:

```
library(tidycensus)
census_api_key("YOUR API KEY", install = TRUE)
```

Or add `CENSUS_API_KEY = "YOUR KEY"` to `.Renviron` via `usethis::edit_r_environ()`

Getting the data

```
library(tidycensus)
# Find variable code:
# v <- load_variables(2022, "acs5"); View(v)

census_vals <- get_acs(
  geography = "tract",
  state = "OR",
  variables = c(med_income = "B06011_001",
                ed_attain  = "B15003_001"),
  year = 2022,
  geometry = TRUE
)
```

```
##
  |
  |
  |
  |===
  |
  |=====
  |
  |=======
  |
  |==========
```

Look at the data

```
census_vals
```

```
## Simple feature collection with 2002 features and 5 fields (with 14 geometries e
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: -124.5662 ymin: 41.99179 xmax: -116.4635 ymax: 46.29083
## Geodetic CRS:  NAD83
## First 10 features:
##          GEOID                                          NAME   variable estima
## 1  41001950100       Census Tract 9501; Baker County; Oregon med_income    300
## 2  41001950100       Census Tract 9501; Baker County; Oregon  ed_attain     21
## 3  41067031912 Census Tract 319.12; Washington County; Oregon med_income    661
## 4  41067031912 Census Tract 319.12; Washington County; Oregon  ed_attain     30
## 5  41067031100    Census Tract 311; Washington County; Oregon med_income    331
## 6  41067031100    Census Tract 311; Washington County; Oregon  ed_attain     22
## 7  41003010300       Census Tract 103; Benton County; Oregon med_income    391
## 8  41003010300       Census Tract 103; Benton County; Oregon  ed_attain     26
## 9  41029002600       Census Tract 26; Jackson County; Oregon med_income    309
## 10 41029002600       Census Tract 26; Jackson County; Oregon  ed_attain     20
```

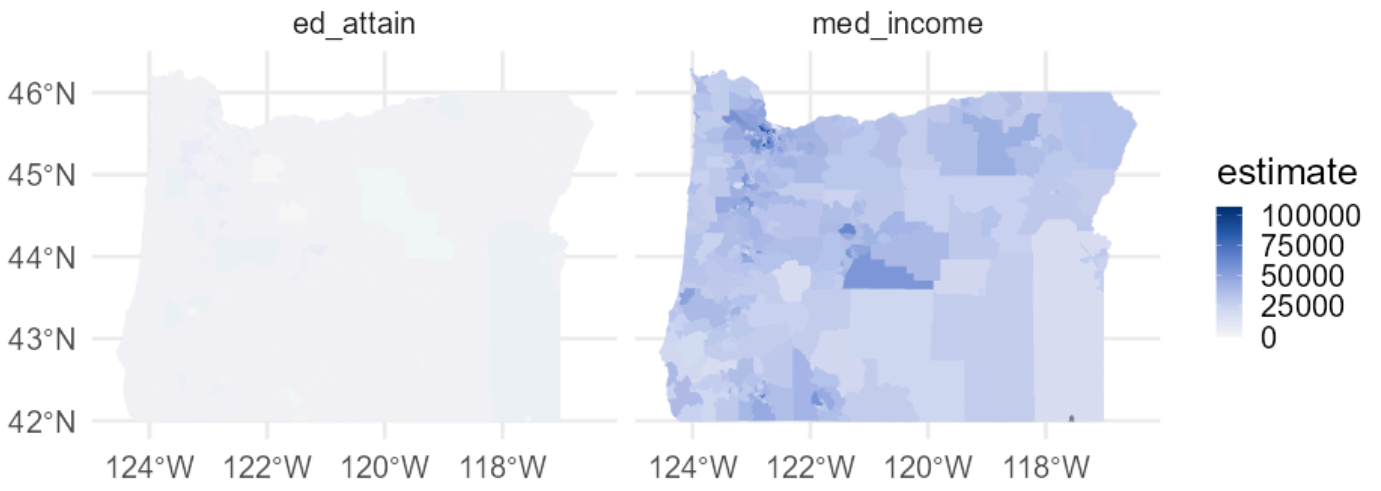
Remove missing geometry rows

```
census_vals <- census_vals[!st_is_empty(census_vals$geometry), ,
```

Plot it

```
library(colorspace)
ggplot(census_vals) +
  geom_sf(aes(fill = estimate, color = estimate)) +
  facet_wrap(~variable) +
  guides(color = "none") +
  scale_fill_continuous_diverging("Blue-Red 3",  rev = TRUE) +
  scale_color_continuous_diverging("Blue-Red 3", rev = TRUE)
```

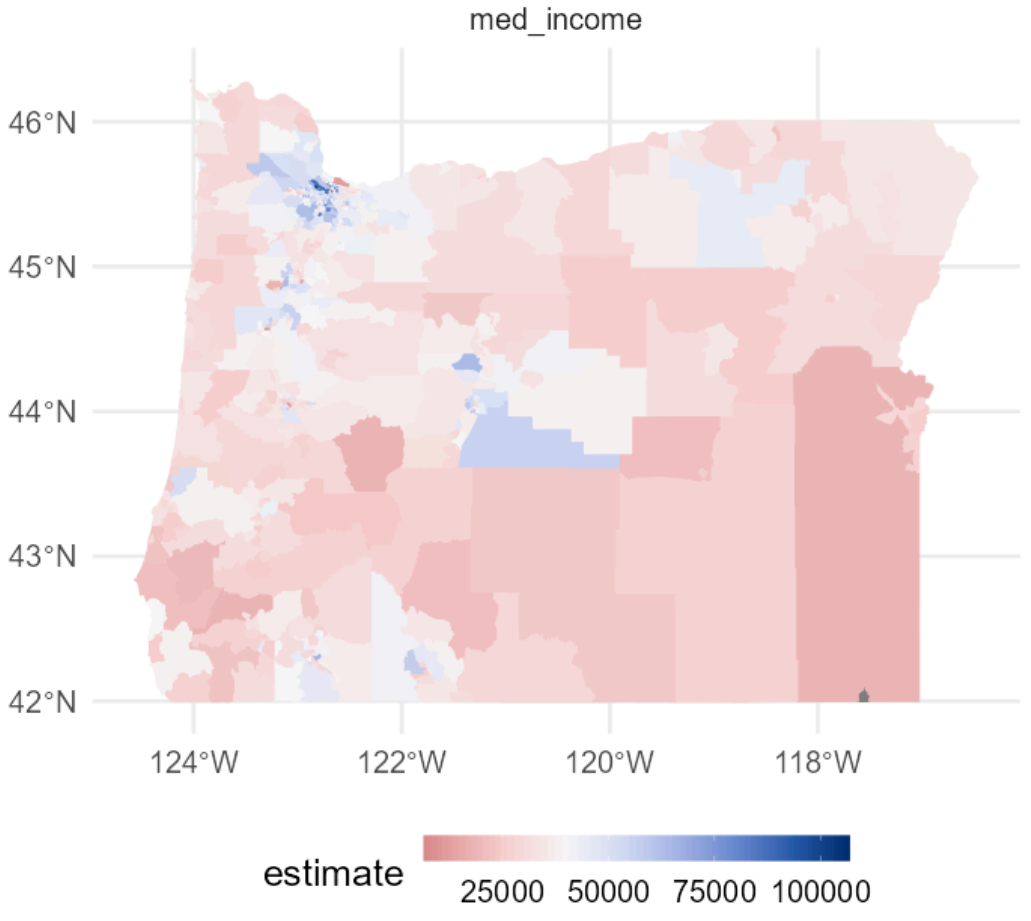
Hmm...


ed_attain
med_income
46°N
45°N
44°N
43°N
42°N
124°W
122°W
120°W
118°W
estimate
100000
75000
50000
25000
0

Try again — set midpoint

```
income <- filter(census_vals, variable == "med_income")

income_plot <- ggplot(income) +
  geom_sf(aes(fill = estimate, color = estimate)) +
  facet_wrap(~variable) +
  guides(color = "none") +
  scale_fill_continuous_diverging(
    "Blue-Red 3", rev = TRUE,
    mid = mean(income$estimate, na.rm = TRUE)
  ) +
  scale_color_continuous_diverging(
    "Blue-Red 3", rev = TRUE,
    mid = mean(income$estimate, na.rm = TRUE)
  ) +
  theme(legend.position = "bottom",
        legend.key.width = unit(2, "cm"))
```

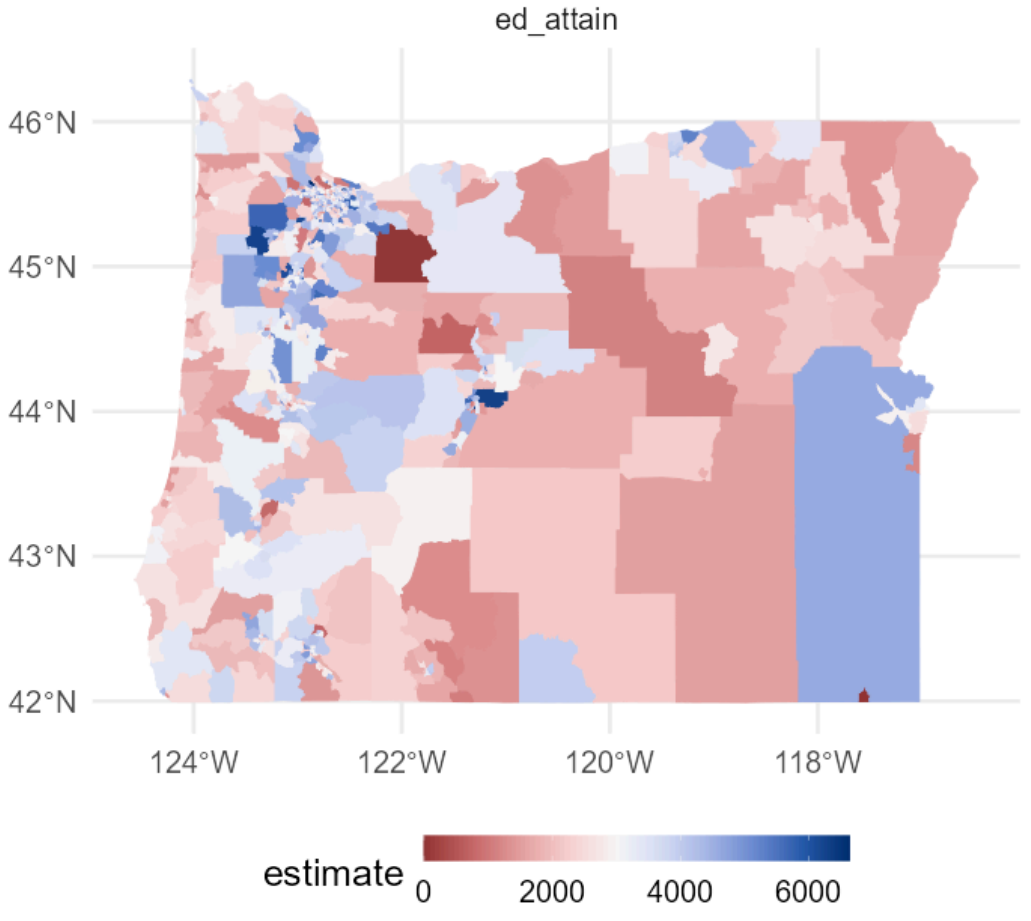
```
income_plot
```


med_income
46°N
45°N
44°N
43°N
42°N
124°W
122°W
120°W
118°W
estimate
25000
50000
75000
100000

Same thing for education

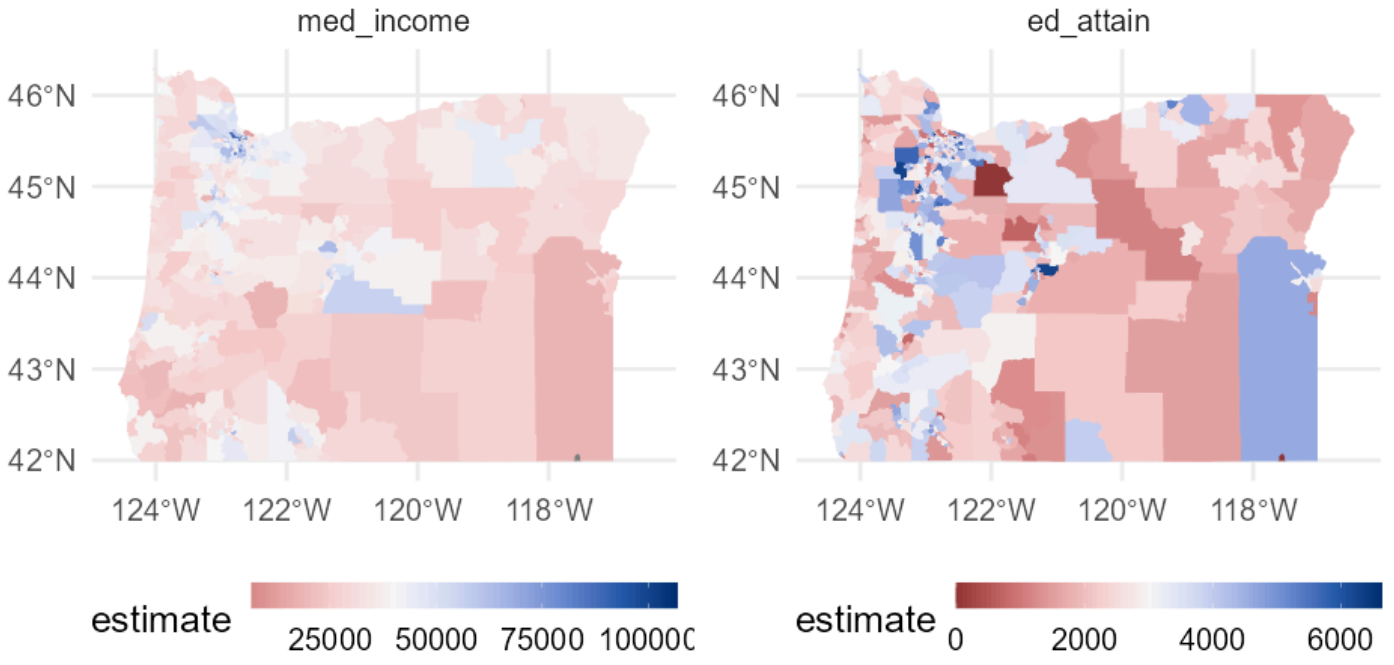
```
ed <- filter(census_vals, variable == "ed_attain")

ed_plot <- ggplot(ed) +
  geom_sf(aes(fill = estimate, color = estimate)) +
  facet_wrap(~variable) +
  guides(color = "none") +
  scale_fill_continuous_diverging(
    "Blue-Red 3", rev = TRUE,
    mid = mean(ed$estimate, na.rm = TRUE)
  ) +
  scale_color_continuous_diverging(
    "Blue-Red 3", rev = TRUE,
    mid = mean(ed$estimate, na.rm = TRUE)
  ) +
  theme(legend.position = "bottom",
        legend.key.width = unit(2, "cm"))
```

```
ed_plot
```


ed_attain
46°N
45°N
44°N
43°N
42°N
124°W
122°W
120°W
118°W
estimate
0
2000
4000
6000

Put them together

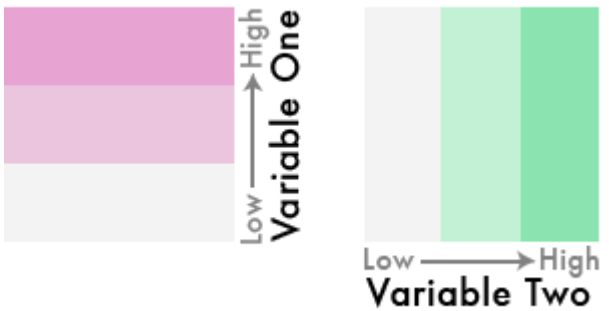
```
gridExtra::grid.arrange(income_plot, ed_plot, ncol = 2)
```



Bivariate color scales

How?

- Break continuous variables into categorical (quartile) bins
- Assign each combination of bins a color
- Make sure the color combinations are perceptually meaningful



gif from Joshua Stevens

Move to wide format

```
wider <- get_acs(
  geography = "tract",
  state = "OR",
  variables = c(med_income = "B06011_001",
                ed_attain  = "B15003_001"),
  year = 2022,
  geometry = TRUE,
  output = "wide"
)

wider <- wider[!st_is_empty(wider$geometry), , drop = FALSE]
```

Find quartiles

```
ed_quartiles <- quantile(
  wider$ed_attainE,
  probs = seq(0, 1, length.out = 4),
  na.rm = TRUE
)

inc_quartiles <- quantile(
  wider$med_incomeE,
  probs = seq(0, 1, length.out = 4),
  na.rm = TRUE
)
```

Create cut variable

```
wider <- wider %>%
  mutate(cat_ed  = cut(ed_attainE,  ed_quartiles),
         cat_inc = cut(med_incomeE, inc_quartiles))

wider %>% select(starts_with("cat"))
```

```
## Simple feature collection with 994 features and 2 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: -124.5662 ymin: 41.99179 xmax: -116.4635 ymax: 46.29083
## Geodetic CRS:  NAD83
## First 10 features:
##                  cat_ed             cat_inc                       geometry
## 1         (0,2.45e+03] (6.53e+03,3.34e+04] MULTIPOLYGON (((-118.5194 4...
## 2  (2.45e+03,3.42e+03] (4.26e+04,1.07e+05] MULTIPOLYGON (((-122.8003 4...
## 3         (0,2.45e+03] (6.53e+03,3.34e+04] MULTIPOLYGON (((-122.8058 4...
## 4  (2.45e+03,3.42e+03] (3.34e+04,4.26e+04] MULTIPOLYGON (((-123.8167 4...
## 5         (0,2.45e+03] (6.53e+03,3.34e+04] MULTIPOLYGON (((-122.7616 4...
## 6         (0,2.45e+03] (6.53e+03,3.34e+04] MULTIPOLYGON (((-122.8811 4...
## 7  (2.45e+03,3.42e+03] (3.34e+04,4.26e+04] MULTIPOLYGON (((-122.8458 4...
## 8  (2.45e+03,3.42e+03] (6.53e+03,3.34e+04] MULTIPOLYGON (((-117.0657 4...
## 9  (3.42e+03,6.64e+03] (3.34e+04,4.26e+04] MULTIPOLYGON (((-123.0464 4...
## 10 (3.42e+03,6.64e+03] (3.34e+04,4.26e+04] MULTIPOLYGON (((-122.906 44...
```

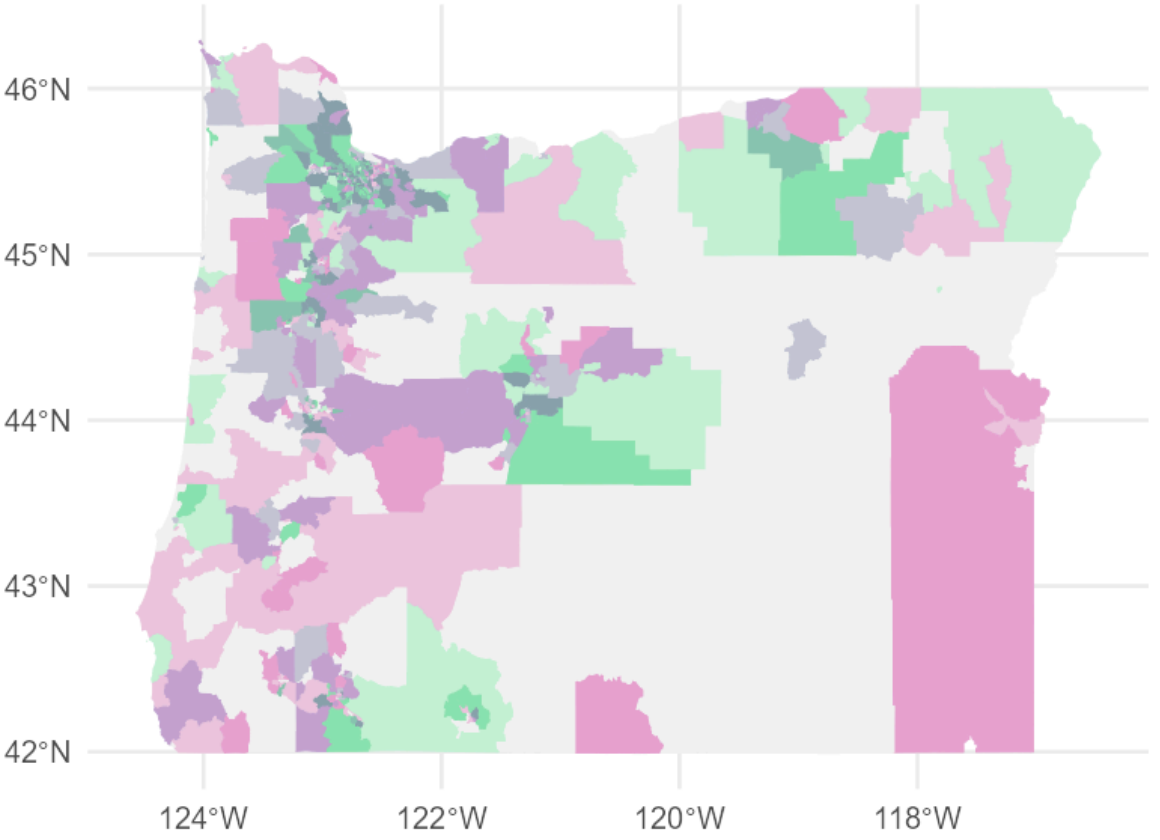
Set palette

```
pal <- st_drop_geometry(wider) %>%
  count(cat_ed, cat_inc) %>%
  arrange(cat_ed, cat_inc) %>%
  drop_na(cat_ed, cat_inc) %>%
  mutate(pal = c("#F3F3F3", "#C3F1D5", "#8BE3AF",
                 "#EBC5DD", "#C3C5D5", "#8BC5AF",
                 "#E7A3D1", "#C3A3D1", "#8BA3AE"))
pal
```

```
##                cat_ed             cat_inc   n     pal
## 1        (0,2.45e+03] (6.53e+03,3.34e+04] 141 #F3F3F3
## 2        (0,2.45e+03] (3.34e+04,4.26e+04]  87 #C3F1D5
## 3        (0,2.45e+03] (4.26e+04,1.07e+05] 100 #8BE3AF
## 4 (2.45e+03,3.42e+03] (6.53e+03,3.34e+04] 114 #EBC5DD
## 5 (2.45e+03,3.42e+03] (3.34e+04,4.26e+04] 110 #C3C5D5
## 6 (2.45e+03,3.42e+03] (4.26e+04,1.07e+05] 107 #8BC5AF
## 7 (3.42e+03,6.64e+03] (6.53e+03,3.34e+04]  75 #E7A3D1
## 8 (3.42e+03,6.64e+03] (3.34e+04,4.26e+04] 133 #C3A3D1
## 9 (3.42e+03,6.64e+03] (4.26e+04,1.07e+05] 123 #8BA3AE
```

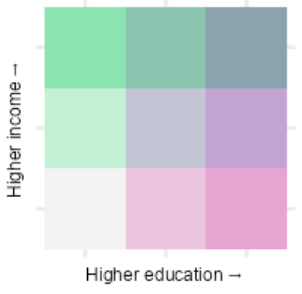
Join and plot

```
bivar_map <- left_join(wider, pal) %>%
  ggplot() +
  geom_sf(aes(fill = pal, color = pal)) +
  guides(fill = "none", color = "none") +
  scale_fill_identity() +
  scale_color_identity()
```


46°N
45°N
44°N
43°N
42°N
124°W
122°W
120°W
118°W

Add a legend

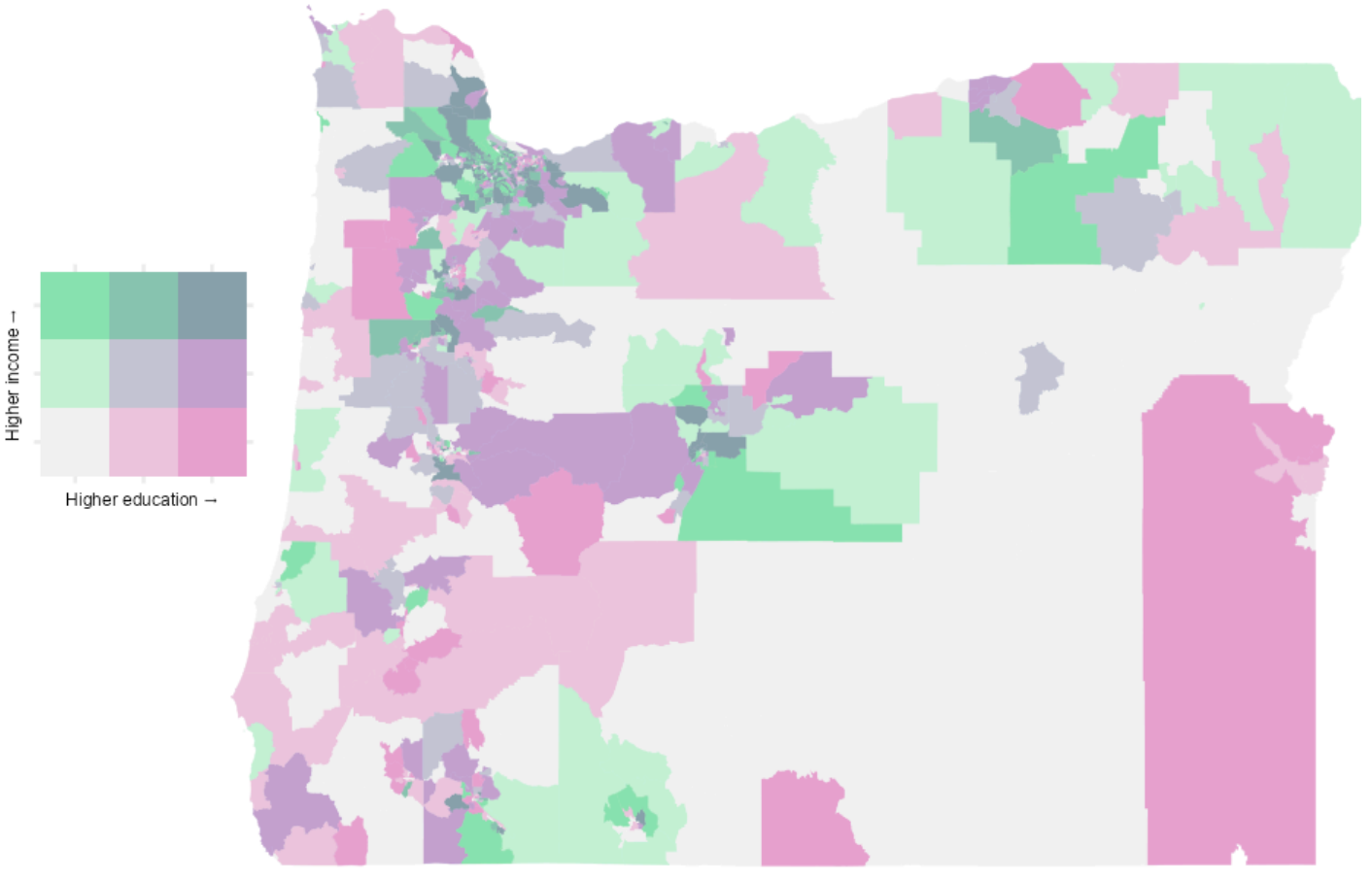
```
leg <- ggplot(pal, aes(cat_ed, cat_inc)) +
  geom_tile(aes(fill = pal)) +
  scale_fill_identity() +
  coord_fixed() +
  labs(x = expression("Higher education" %->% ""),
       y = expression("Higher income"    %->% "")) +
  theme(axis.text = element_blank(),
        axis.title = element_text(size = 12))
leg
```



Put together

```
library(cowplot)
ggdraw() +
  draw_plot(bivar_map + theme_void(), 0.1, 0.1, 1, 1) +
  draw_plot(leg, -0.05, 0, 0.3, 0.3)
```

Coordinates are mostly guess/check depending on aspect ratio.


Higher income →
Higher education →

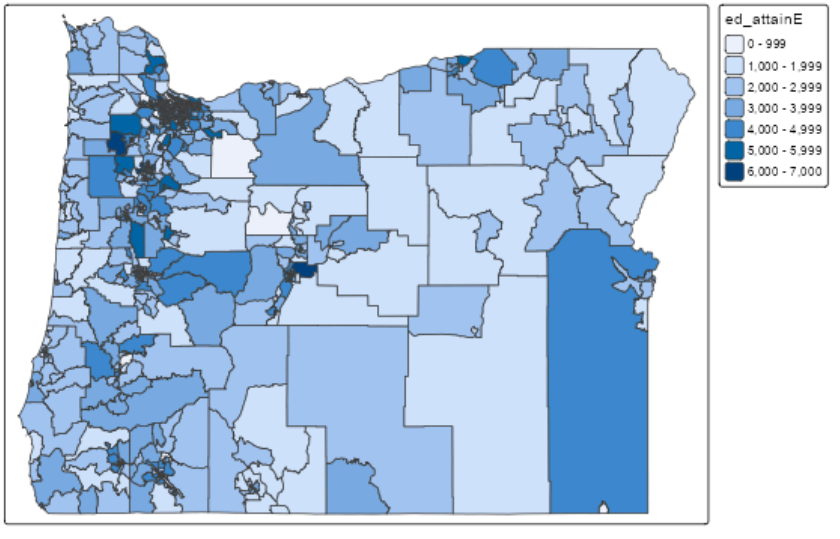
{tmap}

Back to just one variable

I mostly use `ggplot()`, but {tmap} is really powerful and has a clean syntax.

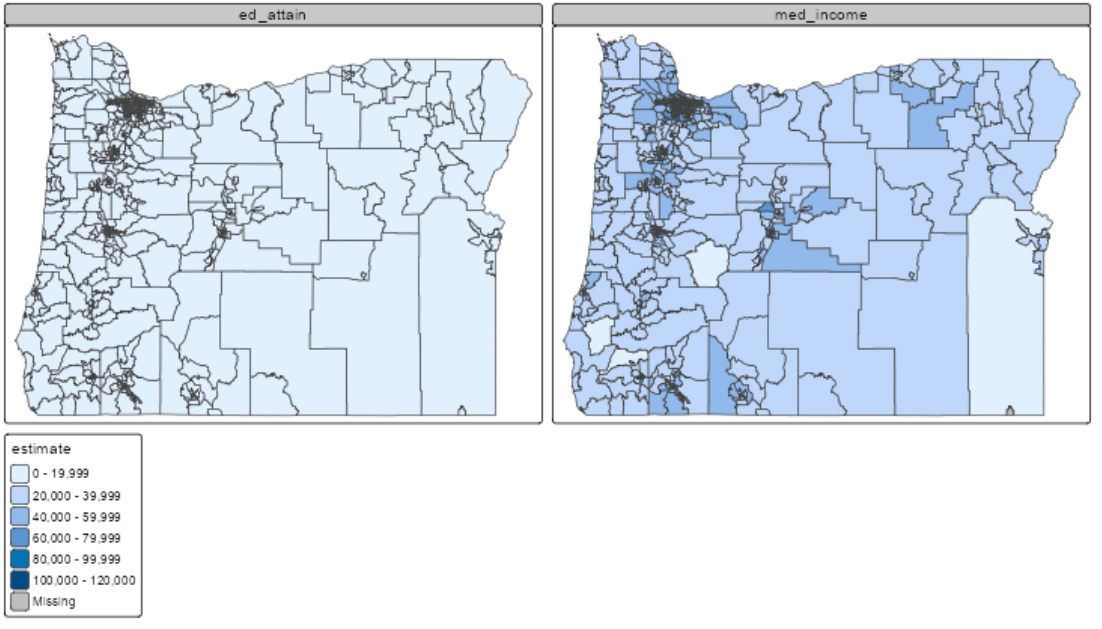
Education map with {tmap}

```
library(tmap)
tm_shape(wider) +
  tm_polygons("ed_attainE") +
  tm_layout(legend.outside = TRUE)
```



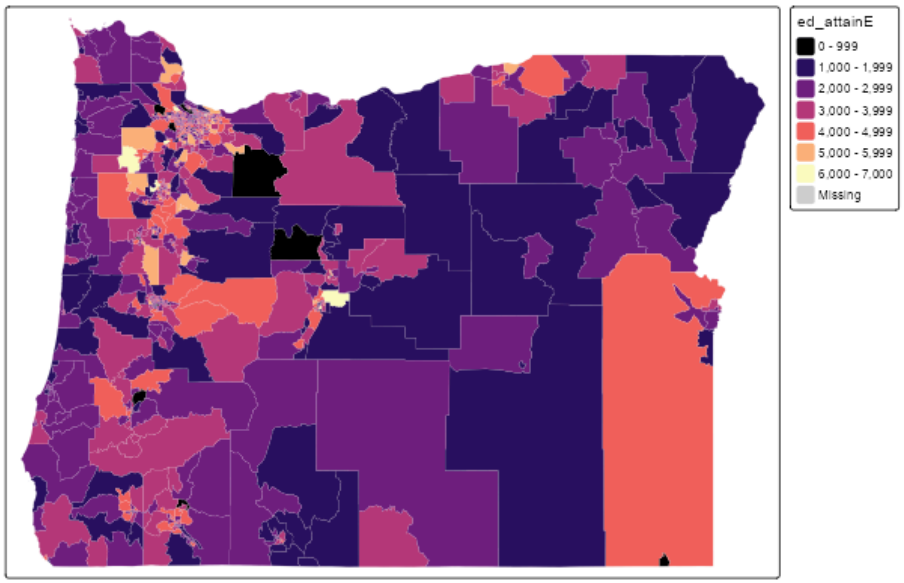
Facet

```
tm_shape(census_vals) +
  tm_polygons("estimate") +
  tm_facets("variable") +
  tm_layout(legend.outside = TRUE)
```



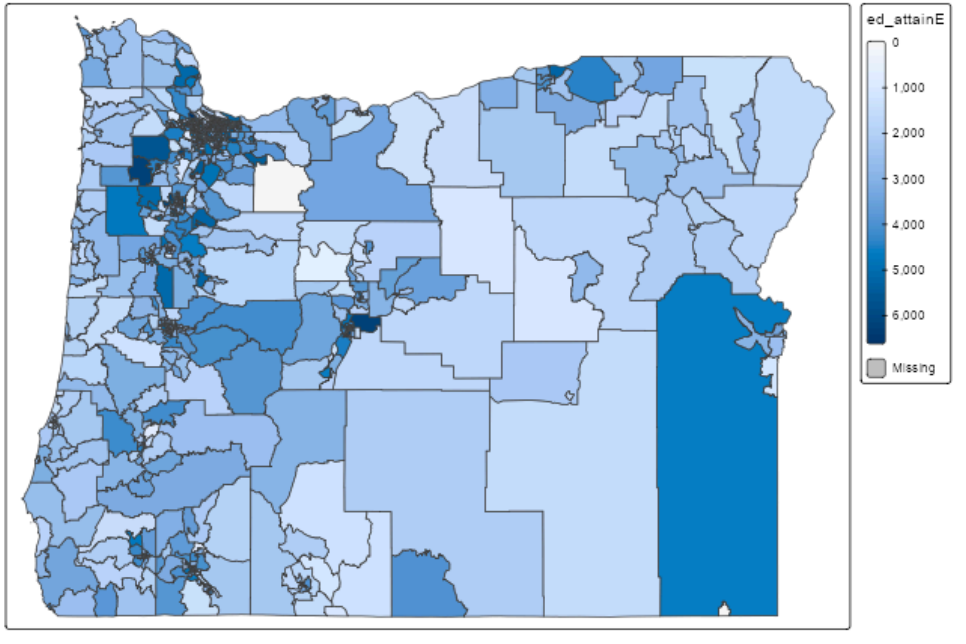
Change colors

```
tm_shape(wider) +
  tm_polygons("ed_attainE",
              palette = "magma",
              border.col = "gray90",
              lwd = 0.1) +
  tm_layout(legend.outside = TRUE)
```



Continuous legend

```
tm_shape(wider) +
  tm_polygons("ed_attainE",
              style = "cont") +
  tm_layout(legend.outside = TRUE)
```



Add text labels

```
cnty <- get_acs(
  geography = "county",
  state = "OR",
  variables = c(ed_attain = "B15003_001"),
  year = 2022,
  geometry = TRUE
)
```

```
##
  |
  |
  |
  |=
  |
  |=
  |
  |==
  |
  |==
  |
  |===
  |
  |====
  |
```

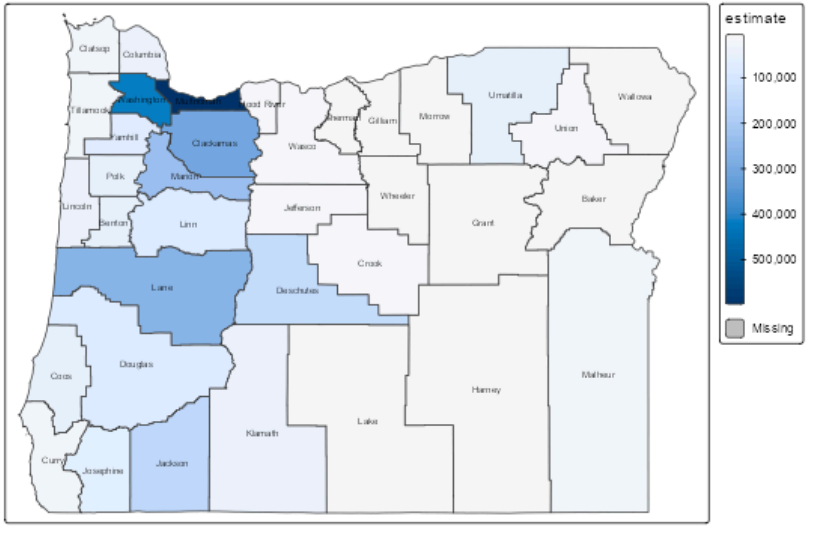
Estimate polygon centroid

```
centroids <- st_centroid(cnty)
```

```
centroids <- centroids %>%
  mutate(county = str_replace_all(NAME, " County, Oregon", ""))
```

Plot with labels

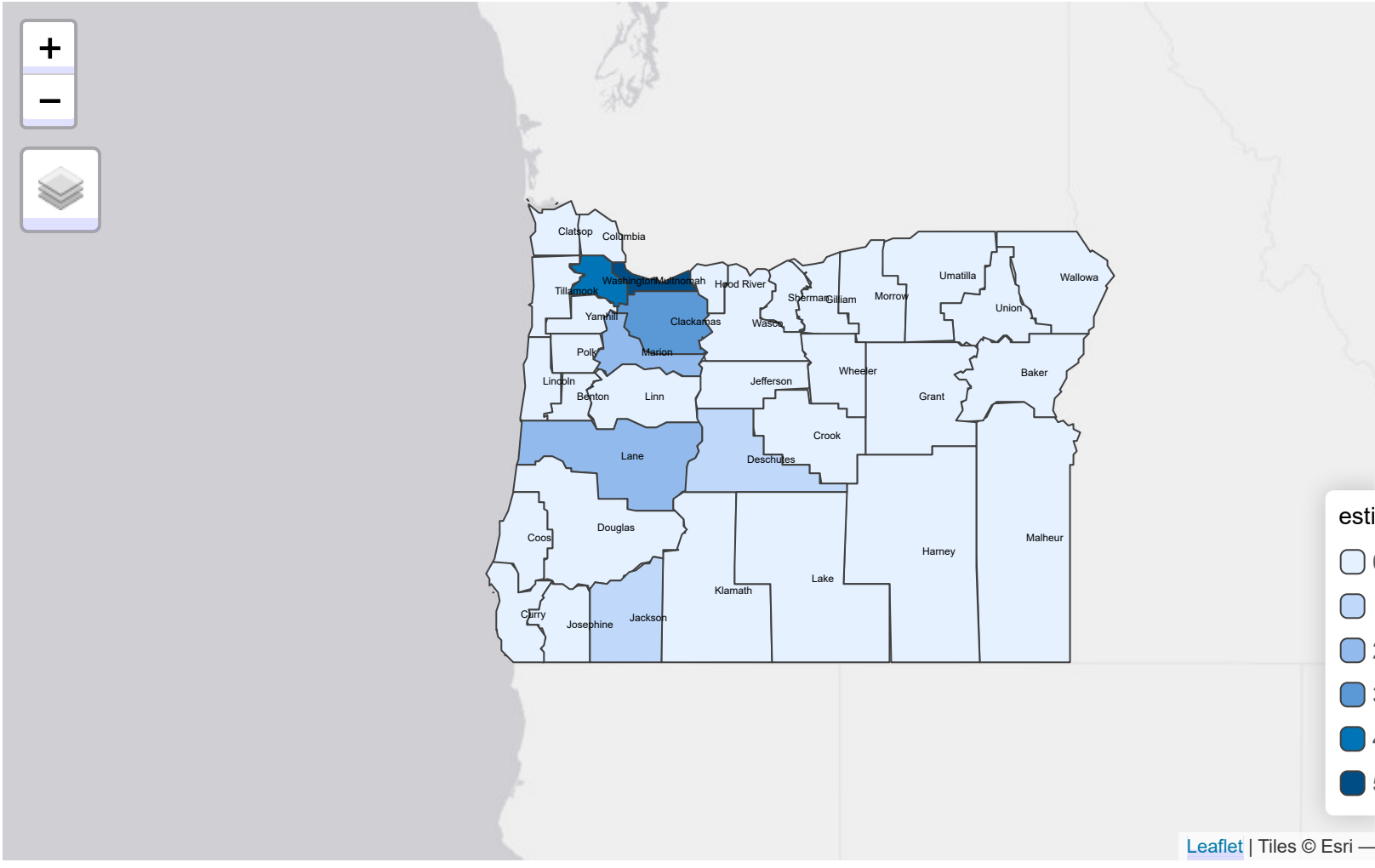
```
tm_shape(cnty) +
  tm_polygons("estimate", style = "cont") +
tm_shape(centroids) +
  tm_text("county", size = 0.5) +
  tm_layout(legend.outside = TRUE)
```



Create interactive maps

Just run `tmap_mode("view")` then reuse the same code:

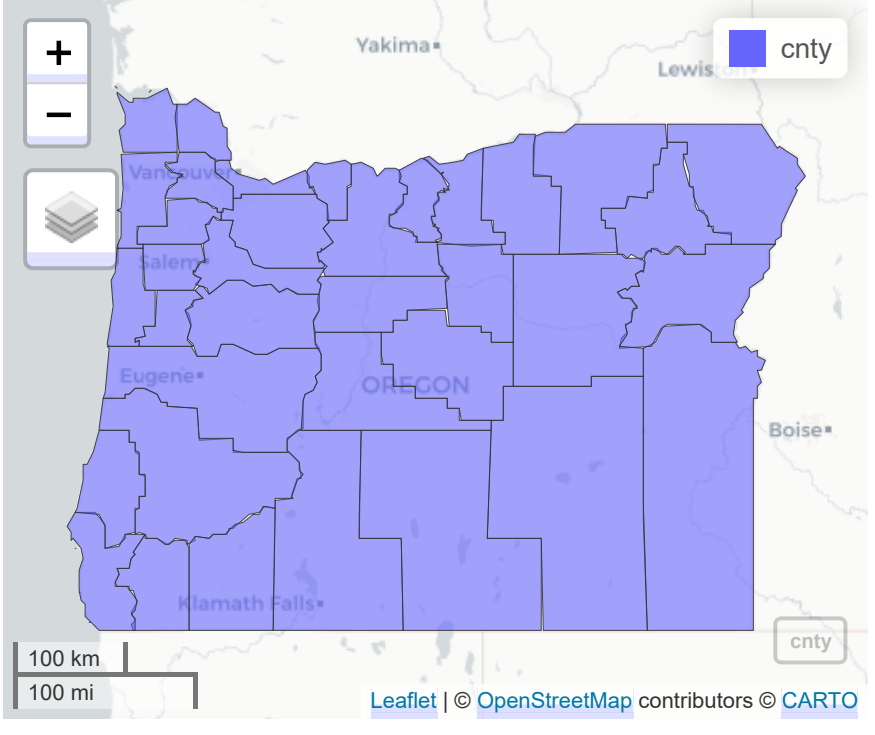
```
tmap_mode("view")

tm_shape(cnty) +
  tm_polygons("estimate") +
tm_shape(centroids) +
  tm_text("county", size = 0.5)
```


Clatsop
Columbia
Tillamook
Washington
Multnomah
Hood River
Sherman
Gilliam
Morrow
Umatilla
Wallowa
Union
Yamhill
Clackamas
Wasco
Polk
Marion
Jefferson
Wheeler
Baker
Lincoln
Benton
Linn
Grant
Crook
Lane
Deschutes
Douglas
Coos
Harney
Malheur
Lake
Klamath
Curry
Josephine
Jackson
esti
Leaflet | Tiles © Esri —

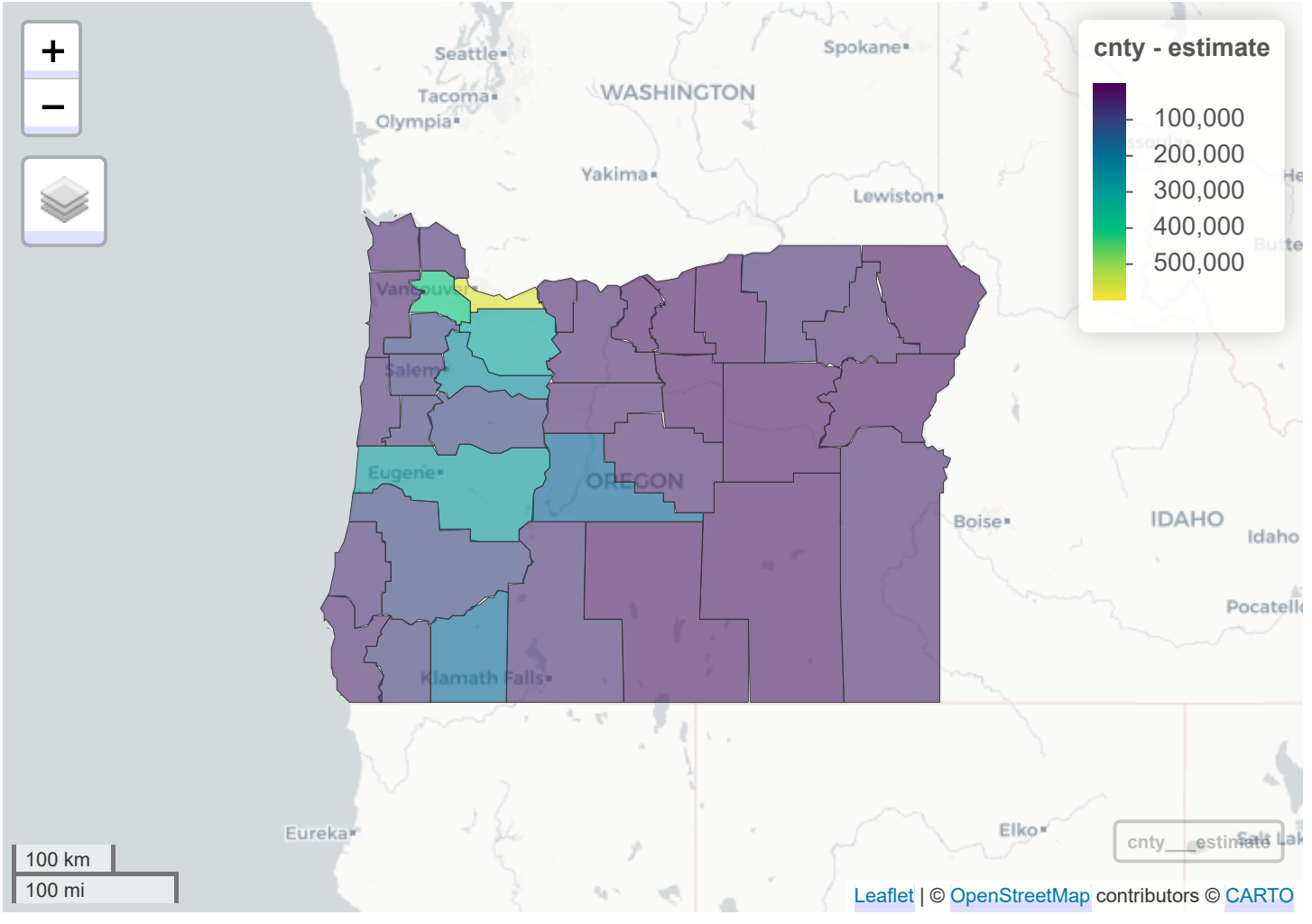
mapview

Really quick easy interactive maps — great for exploratory work.

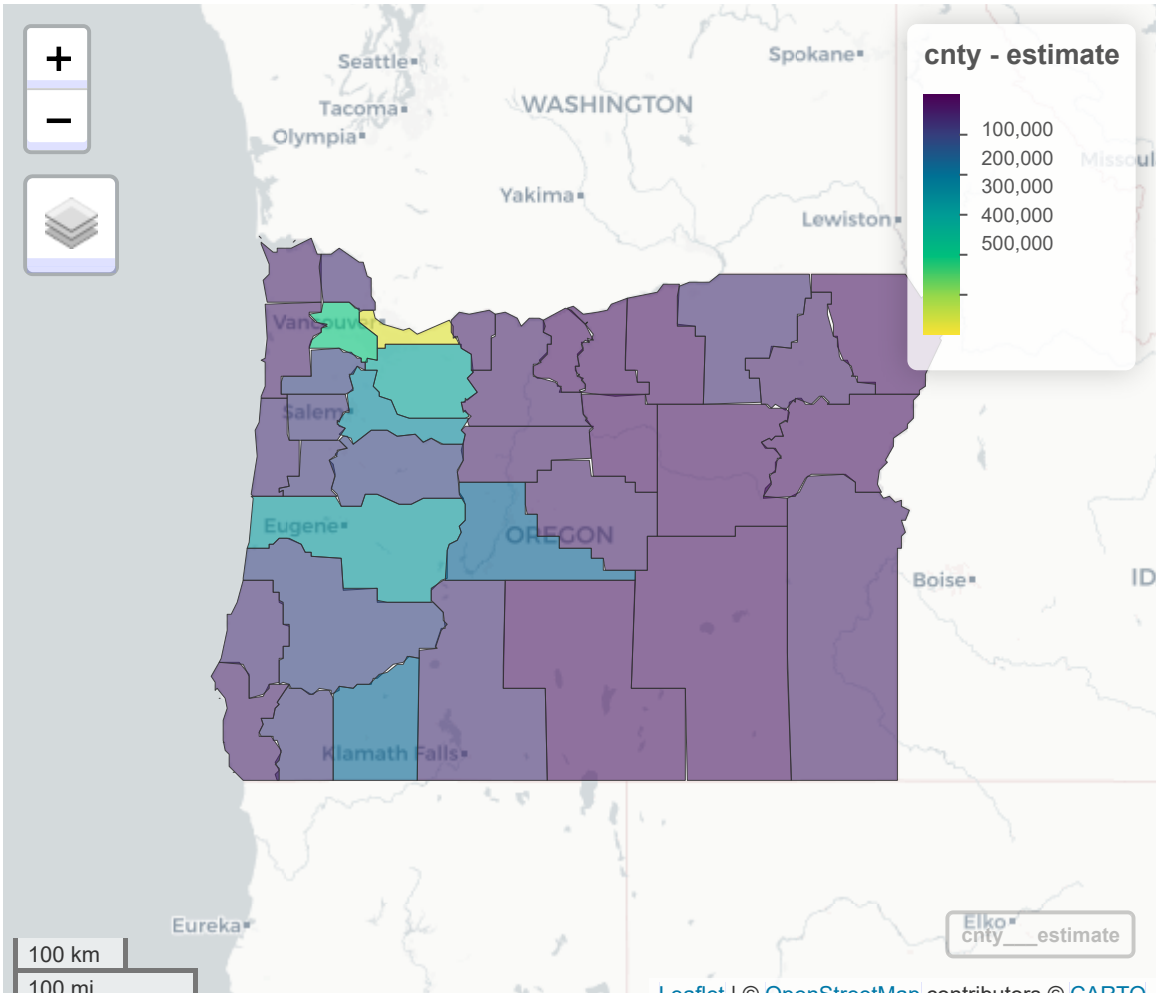
```
library(mapview)
mapview(cnty)
```



```
mapview(cnty, zcol = "estimate")
```



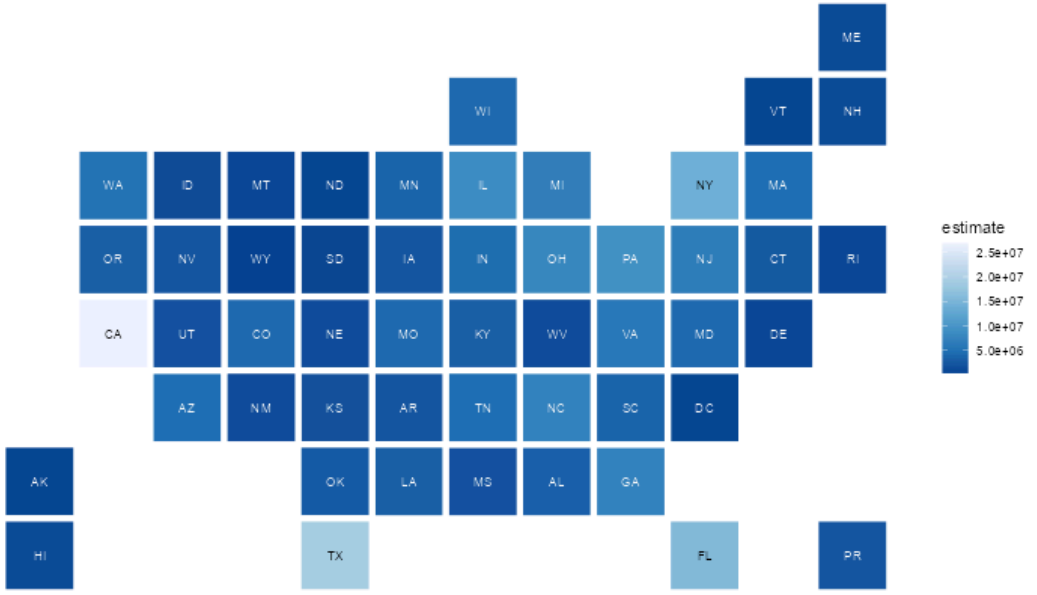
```
mapview(cnty,
        zcol = "estimate",
        popup = leafpop::popupTable(cnty,
                                    zcol = c("NAME", "estimate"))
```



A few other things of note

statebins

```
library(statebins)
statebins(states,
          state_col = "NAME",
          value_col = "estimate") +
  theme_void()
```

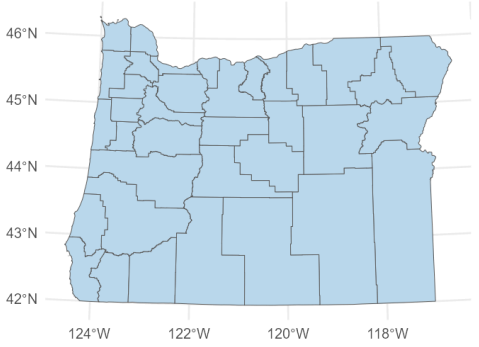


Cartograms

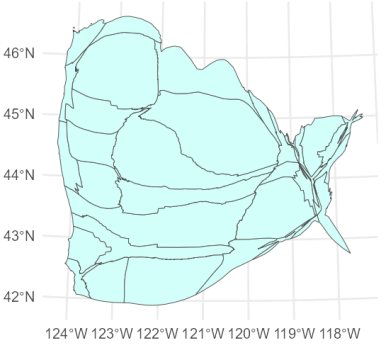
```
library(cartogram)
or_county_pop <- get_acs(
  geography = "county",
  state = "OR",
  variables = "B01003_001",
  year = 2022,
  geometry = TRUE
)

or_county_pop  <- st_transform(or_county_pop, crs = 2992)
carto_counties <- cartogram_cont(or_county_pop, "estimate")
```

Compare

```
ggplot(or_county_pop) +
  geom_sf(fill = "#BCD8EB")
```




```
ggplot(carto_counties) +
  geom_sf(fill = "#D5FFFA")
```



US cartogram by population

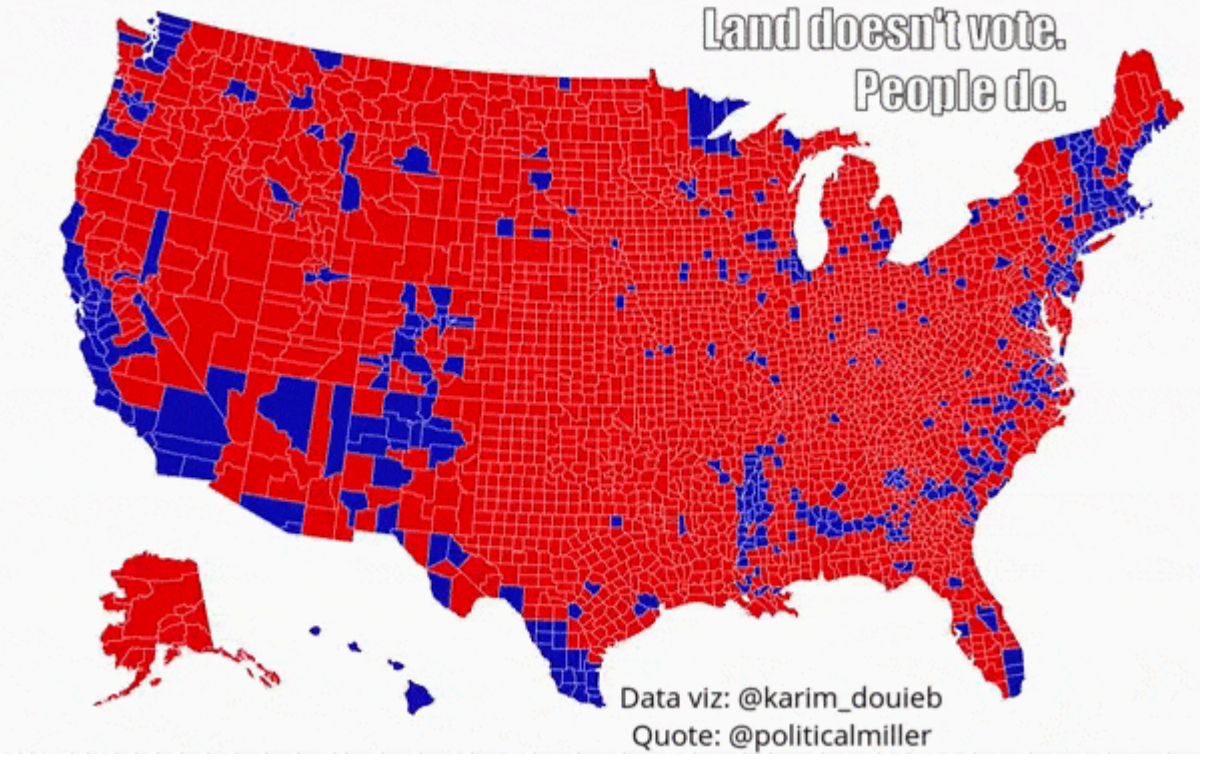
```
state_pop <- get_acs(
  geography = "state",
  variables = "B01003_001",
  year = 2022,
  geometry = TRUE
)

state_pop     <- st_transform(state_pop, crs = 2163)
carto_states  <- cartogram_cont(state_pop, "estimate")
```

```
ggplot(carto_states) +
  geom_sf()
```

Last note

You may or may not like cartograms.

Just be careful not to lie with maps.



Next time:
Shiny App+
Some final presentations!